Đề xuất Kiến trúc StructPaper v2

https://gemini.google.com/app/22f029891cb88752

User prompt: StructPaper v2 — Đề xuất kiến trúc & design pattern Căn cứ: Yêu_cầu_phần_mềm.docx, Template_EN.docx, Template_VI.docx, DOCS_HIEN_TRANG_UPLOAD_EDITOR.md, DOCS_KIEN_TRUC_LUONG_DU_LIEU_UPLOAD_EDITOR.md và PDF mẫu ID630_HS. 0. Tóm tắt điều hành Hiện trạng đã chạy được một đường ống trọn vẹn Upload → CompletePaper → Editor → LaTeX → PDF. Nhưng ba thứ đang dính chặt vào nhau nên nhiều người không thể làm song song: 1. Logic đọc DOCX nằm trong một file hơn 2.100 dòng (docx-parser.ts). 2. Mô hình dữ liệu quá phẳng (chuỗi text, ảnh base64, số thứ tự lưu cứng). 3. UnifiedEditorView điều phối mọi thứ (state, lưu, biên dịch, dàn trang). Mọi yêu cầu lớn trong file yêu cầu (cross-reference, đánh số lại equation, renumber citation, JATS XML, nhiều journal, revision, preflight, batch) đều cần một mô hình tài liệu có cấu trúc và ID ổn định. Vì vậy đó là thứ phải làm đầu tiên. Bảy quyết định lớn được đề xuất: 1. Tách lõi miền (doc-model) thuần TypeScript, không dính React, DOM, IndexedDB hay HTTP. 2. Manuscript v2: cây block + inline node + ID ổn định; số Figure/Table/Equation/Reference là dữ liệu dẫn xuất, không lưu cứng. 3. Ingest DOCX là pipeline nhiều tầng: ưu tiên đọc theo style Word của template, sau đó heuristic, cuối cùng mới AI. Chạy trong Web Worker và viết dạng isomorphic để dùng lại cho batch phía server. 4. Journal Profile là dữ liệu (JSON có schema), không phải code. Thêm tạp chí mới = thêm profile và một file .cls. 5. Editor = Store + Command + Feature module + Block registry (undo/redo, autosave, op-log đồng bộ BE có sẵn). 6. Persistence offline-first: IndexedDB (Dexie) đứng sau interface Repository; sau này Spring Boot/MySQL chỉ là thêm một adapter. 7. Compile service tách riêng, sandbox, có cache/queue/cancel; dùng chung cho PDF preview và PDF xuất bản (PDF/X). 8. Phát hiện từ tài liệu (các bất cập cần xử lý) # Phát hiện Bằng chứng trong tài liệu Hệ quả Hướng xử lý 1 Parser một cục docx-parser.ts >2.100 dòng gồm mammoth, JSZip, OMML, metadata, bảng, ảnh, reference, nhận diện journal, merge AI Không chia việc được; sửa bảng có thể vỡ ảnh; khó test Pipeline nhiều tầng (mục 4) 2 Parse XML bằng regex Mô tả dùng regex + DOM; ommlToLatex chỉ có 6 cấu trúc (f, sSup, sSub, sSubSup, rad, d) Vỡ với matrix, tổng/tích phân, accent, MathType, equation array (yêu cầu mục 4 nêu rõ) Parser XML thật; OMML → MathML → LaTeX; giữ OMML gốc 3 Bỏ phí style của template Template_EN/VI có ~35 style đặt tên: ArticleTitle, Authors, Affiliation, CorrespondingAuthor, Date, Abstract, Keywords, Heading1-3, Equation, TableCaption, TableNote, SourceLine, FigureCaption, Section*, Reference. Các style Guide*, Instruction phải bị bỏ qua Tác giả đã dùng đúng template mà parser vẫn đoán bằng heuristic/AI; sai và tốn token Recognizer theo style, độ tin cậy 1.0 4 Model phẳng content: string chứa [1, 2] thô; không có inline format; PaperSection không có equation/list/footnote; hình/bảng không có source; không có block cho Ethics/Funding/Contributions/Conflict/Data availability (template có đủ) Mất in nghiêng (H. pylori trong PDF mẫu), chỉ số dưới (FEV_1); không cross-ref; renumber bằng regex Manuscript v2 (mục 3) 5 Hai biểu diễn bảng headers/rows cùng headerRows; normalizePaperTable đoán header 2 tầng từ chuỗi kiểu A / B Dữ liệu suy đoán, dễ sai, khó làm longtable, bảng ngang, repeat header Một biểu diễn lưới (cell có rowSpan/colSpan) 6 Ảnh là base64 trong JSON src: data:...; ép mime png/jpeg; lưu sessionStorage (~5 MB); catch {} nuốt lỗi; gửi lại base64 mỗi lần compile; alias fig1.png/fig1.jpg Bài nhiều ảnh mất dữ liệu im lặng khi lưu; EMF/WMF/TIFF/EPS/PDF (yêu cầu) bị gán nhầm mime; không kiểm tra DPI Asset store theo hash nội dung 7 .doc UI nhận .doc nhưng JSZip/mammoth không đọc được .doc nhị phân (OLE2) Lỗi khi người dùng chọn .doc Chuyển đổi phía server (LibreOffice) hoặc từ chối rõ ràng 8 State 2 tầng + sessionStorage formData ↔ PaperContext; isDirty set tay; lưu bằng Ctrl+S Không undo/redo, không autosave, không lịch sử revision, đóng tab là mất Store + Command + IndexedDB 9 Hai bộ sinh LaTeX exporters.ts (exportToLatex) và latex-template.ts (generateFullJournalLatex) Trùng lặp, dễ lệch Một LaTeX emitter duy nhất 10 Compile ở hai nơi, chưa sandbox Next route và Spring cùng compile; thay font bằng string replace; chạy xelatex trên TeX do client gửi Rủi ro \write18/\input đọc file hệ thống; không queue/cancel/cache Compiler service (mục 8) 11 Lộ API key, lộ bản thảo NEXT_PUBLIC_OPENAI_API_KEY, người dùng gõ sk-… Khóa nằm ở client; bản thảo chưa công bố đi thẳng ra ngoài AI qua proxy BE, có consent 12 Quy tắc journal rải rác và mâu thuẫn Tóm tắt 150–250 từ (app) vs 275–500 từ (template); từ khóa dấu phẩy vs chấm phẩy; ngày 02-10-2026 vs dd/mm/yyyy; HS có 8 loại bài vs template 4 loại; trích dẫn [1] (template) vs chỉ số trên (PDF mẫu) Mỗi nơi một con số Journal Profile là nguồn sự thật duy nhất 13 Song ngữ cứng Model có title/titleEn; template lại tách thành 2 file EN và VI Không mở rộng được lang chính + translations 14 PDF mẫu có lỗi dữ liệu DOI ghi …v7i1.630 nhưng bài là 7(2) Chính là loại lỗi preflight phải bắt Rule QC đối chiếu DOI với volume/issue; dùng PDF mẫu làm golden test thị giác 2. Kiến trúc tổng thể 2.1 Phân tầng (Hexagonal / Ports & Adapters) apps/web (Next.js, vỏ mỏng: route, layout, DI) │ packages/feature-* UI theo tính năng (metadata, authors, body, references, qc, layout, preview, export) │ packages/editor-core store, command bus, history, block registry, autosave │ packages/ingest-docx · exporters · qc-engine · journal-profiles · ai-client · storage · compiler-client │ packages/doc-model types, zod schema, numbering, visitors, migrations ← không phụ thuộc gì apps/api (Spring Boot 3, modular monolith) services/compiler-worker (TeX Live/Tectonic, sandbox) Luật phụ thuộc: mũi tên chỉ đi xuống. doc-model không import gì; ingest, exporters, qc chỉ phụ thuộc doc-model. Ép luật bằng dependency-cruiser hoặc eslint-plugin-boundaries trong CI; vi phạm thì fail build. Đây là điều kiện để nhiều người làm song song không đạp lên nhau. 2.2 Cổng (port) và bộ chuyển đổi (adapter) Port (interface) Adapter giai đoạn IndexedDB Adapter giai đoạn Spring Boot ManuscriptRepository IndexedDbManuscriptRepo (Dexie) HttpManuscriptRepo + decorator SyncedRepo AssetRepository Blob trong IndexedDB Upload qua API → S3/MinIO hoặc filesystem CompilerPort LocalCliCompiler (chỉ dev) RemoteCompiler (job qua API) AiPort MockAi / bật tắt Proxy BE → OpenAI JournalProfileProvider JSON đóng gói sẵn API /journals có version DocConverterPort (.doc → .docx) không hỗ trợ, báo lỗi rõ LibreOffice headless phía server Clock, IdGenerator thật giả (để test ổn định) UI và core chỉ biết interface. Đổi nơi lưu hoặc nơi biên dịch không phải sửa Editor. 2.3 Cấu trúc monorepo (pnpm workspaces + Turborepo) structpaper/ ├─ apps/ │ ├─ web/ # Next.js: chỉ routing + lắp ráp (composition root) │ └─ api/ # Spring Boot ├─ services/ │ └─ compiler-worker/ # Docker: texlive-xetex + fonts, chạy job biên dịch ├─ packages/ │ ├─ doc-model/ # Manuscript v2, zod, numbering, visitor, migrations │ ├─ ingest-docx/ # pipeline đọc Word (mục 4) │ ├─ journal-profiles/ # JSON + schema + loader (mục 5) │ ├─ exporters/ # latex, docx, jats, bibtex, zip, html │ ├─ qc-engine/ # rule engine + rules (mục 7) │ ├─ editor-core/ # store, commands, history, block registry │ ├─ storage/ # repository interfaces + dexie + http + sync │ ├─ compiler-client/ # CompilerPort + client cho API │ ├─ ai-client/ # AiPort + prompt có version │ ├─ feature-metadata/ feature-authors/ feature-abstract/ │ ├─ feature-body/ # heading, paragraph, figure, table, equation │ ├─ feature-references/ feature-qc/ feature-layout/ feature-preview/ feature-export/ │ └─ ui-kit/ # Select, DatePicker, Modal, Dock... ├─ fixtures/ # docx mẫu + JSON kỳ vọng + PDF tham chiếu (golden) ├─ contracts/ # openapi.yaml + JSON Schema sinh từ zod └─ CODEOWNERS # chia quyền sở hữu theo package 3. Mô hình dữ liệu Manuscript v2 (doc-model) 3.1 Nguyên tắc - ID ổn định cho mọi block/inline đích tham chiếu (ULID). Cross-reference và citation trỏ bằng ID, không bằng số. - Số thứ tự là dữ liệu dẫn xuất. Figure 1, Table 2, Equation (3), [1]…[n] được tính bằng hàm computeNumbering(doc) từ vị trí và thứ tự xuất hiện lần đầu. Xóa/chèn giữa bài thì mọi thứ tự cập nhật theo; production editor không sửa tay (đúng yêu cầu mục 4 và 5). - Nội dung văn bản là cây inline,*

không phải chuỗi: giữ được đậm/nghiêng/chỉ số, citation, cross-ref, công thức nội dòng. - Ảnh không nằm trong JSON; chỉ có assetId = sha256. - Có schemaVersion + migration (vì dữ liệu nằm lâu dài trong IndexedDB người dùng và MySQL). - Dữ liệu nguồn gốc (provenance) được giữ lại: block nào lấy từ đâu, độ tin cậy bao nhiêu, để UI tô vàng chỗ cần người kiểm. 3.2 Hình dạng (rút gọn) type Id = string; // ULID interface Manuscript { schemaVersion: 2; id: Id; lang: 'vi' | 'en'; // ngôn ngữ chính của bài (khớp Template_VI/EN) journalId: string; articleType: string; // lấy từ JournalProfile.articleTypes meta: FrontMatter; body: Section[]; // cây section (tương ứng <sec> của JATS) back: Statement[]; // ethics, acknowledgments, funding, contributions, conflict, data references: Reference[]; // cấu trúc hóa, không chỉ chuỗi thô assets: Record<string, AssetMeta>; // sha256 -> {mime, w, h, dpi, vector} layoutOverrides?: LayoutOverrides; // chỉ phần khác profile provenance?: Record<Id, Provenance>; } interface FrontMatter { title: Inline[]; translations?: { en?: Inline[] }; authors: Author[]; // {id, name, affiliationIds, corresponding, email, orcid} affiliations: Affiliation[]; // {id, text, translations} abstract: Inline[][]; // 1 đoạn hoặc có cấu trúc keywords: string[]; dates: { received?; revised?; accepted?; published? }; // ISO 8601 bên trong, định dạng theo profile khi hiển thị doi?: string; volume?: string; issue?: string; pages?: [number, number]; } type Block = | { type: 'paragraph'; id: Id; content: Inline[] } | { type: 'list'; id: Id; ordered: boolean; items: Block[][] } | { type: 'quote' | 'box'; id: Id; content: Block[] } | { type: 'equation'; id: Id; latex: string; omml?: string; numbered: boolean; label?: string; status: 'ok' | 'fallback-image' } | { type: 'figure'; id: Id; assetId: string; caption: Inline[]; source?: Inline[]; panels?: Panel[]; layout: FigureLayout } | { type: 'table'; id: Id; caption: Inline[]; grid: Cell[][]; headerRows: number; notes?: Inline[]; source?: Inline[]; layout: TableLayout } | { type: 'pageBreak'; id: Id }; interface Section { id: Id; level: 1 | 2 | 3; title: Inline[]; blocks: Block[]; children: Section[] } type Inline = | { t: 'text'; v: string; marks?: ('b' | 'i' | 'sup' | 'sub' | 'code')[] } | { t: 'cite'; refIds: Id[] } // hiển thị [1], [2–4] hoặc chỉ số trên theo profile | { t: 'xref'; target: Id; kind: 'figure' | 'table' | 'equation' | 'section' } | { t: 'math'; latex: string } | { t: 'footnote'; blocks: Block[] } | { t: 'link'; href: string; content: Inline[] }; interface Cell { content: Inline[]; rowSpan: number; colSpan: number; align?: 'l' | 'c' | 'r'; header?: boolean } interface Reference { id: Id; raw: string; csl?: CslJson; doi?: string; url?: string; parse: 'ok' | 'partial' | 'failed' } 3.3 Dữ liệu dẫn xuất (không lưu) computeNumbering(doc) → { figure: Map<Id, number>, table: Map<Id, number>, equation: Map<Id, number>, citationOrder: Id[], // thứ tự xuất hiện lần đầu → số [n] uncitedRefs: Id[], danglingCites: Id[], xrefBroken: Id[], firstMentionBeforeFloat: Map<Id, boolean> // template yêu cầu nhắc trước khi hình/bảng xuất hiện } Hàm này là pure function, memoize theo phiên bản document, dùng chung cho editor, QC và mọi exporter. Đây là lời giải cho ví dụ xóa Reference 5 hoặc chèn Equation giữa bài. 3.4 Di chuyển từ CompletePaper v1 Viết migrateV1toV2(CompletePaper): Manuscript (tách [1,2] trong chuỗi thành node cite, tách base64 thành asset, chuyển headerRows thành lưới). Giữ adapter ngược tạm thời để UI cũ vẫn chạy trong lúc chuyển (xem lộ trình, mục 14). 4. Pipeline ingest DOCX (packages/ingest-docx) Ý tưởng: tách đọc (lossless) khỏi hiểu (semantic). Mỗi tầng là một thư mục, một nhóm người, kiểm thử độc lập. .docx —▶ [1 Package] —▶ [2 OOXML→RawDoc] —▶ [3 Recognizers] —▶ [4 Assemblers] —▶ [5 Normalizers] —▶ [6 Validate] —▶ Manuscript + IngestReport Tầng Việc Pattern Ghi chú 1. Package Mở ZIP, đọc các part: document.xml, styles.xml, numbering.xml, footnotes.xml, endnotes.xml, comments.xml, _rels, media/ Adapter Chỉ phụ thuộc thư viện ZIP; trả về DocxPackage 2. OOXML → RawDoc Duyệt XML một lần bằng parser thật (isomorphic, ví dụ fast-xml-parser, không dùng DOMParser để chạy được cả Node). Giải basedOn của style. Gỡ w:sdt, mc:AlternateContent. Xử lý field (fldChar/instrText: REF, SEQ, HYPERLINK, ADDIN EN.CITE, ADDIN CSL_CITATION của Zotero/Mendeley) Visitor Kết quả là RawBlock[] (đoạn + run + style + numbering + bảng lưới + drawing ref + math) và chưa có ngữ nghĩa 2b. Track Changes Chính sách acceptAll (mặc định) / rejectAll / keepAsComments cho w:ins, w:del, w:moveFrom/To Strategy Đáp ứng yêu cầu Input: Word có Track Changes 3. Recognizers Mỗi recognizer nhìn một RawBlock + ngữ cảnh và trả về claim {role, confidence, evidence} Chain of Responsibility + Strategy Xem bên dưới 4. Assemblers Gom các block đã gán vai trò thành từng khối miền Builder FrontMatterAssembler, AbstractAssembler, BodyAssembler (dựng cây section), TableAssembler, FigureAssembler, EquationAssembler, ReferenceAssembler, BackmatterAssembler, FootnoteAssembler 5. Normalizers Hàm thuần Manuscript → Manuscript, chạy theo thứ tự Pipeline UnicodeNFC (bắt buộc cho tiếng Việt: NFD và NFC dễ lẫn), VietnameseSpacing (cái đang là normalizeVietnameseWordSpacing), EndNoteCleanup, Punctuation, CitationLinker, XRefLinker, CaptionNumberStripper 6. Validate Kiểm bằng zod, sinh IngestReport: số block không nhận diện được, claim độ tin cậy thấp, cảnh báo Specification Report đi vào ngăn kéo nội dung bóc tách 4.1 Recognizer theo thứ tự ưu tiên Ưu tiên Recognizer Cách làm Độ tin cậy 1 StyleMapRecognizer Dùng JournalProfile.styleMap (ví dụ ArticleTitle→title, Authors→authors, Abstract→abstract, Heading1→h1, SourceLine→source, Reference→reference, Guide*/Instruction→bỏ qua) 1.0 2 StructureRecognizer Theo vị trí và mẫu: caption Table N. / Bảng N. / Figure N. / Hình N., Keywords: / Từ khóa:, dòng [n] trong danh mục, tiêu đề References / Tài liệu tham khảo 0.8 3 FormatHeuristicRecognizer Cỡ chữ, in đậm, căn giữa, đoạn đầu tài liệu → title/authors 0.5–0.6 4 AiRecognizer (tùy chọn) Chỉ gọi cho phần front-matter và các block còn mơ hồ (độ tin cậy < ngưỡng); trả về đề xuất chứ không ghi đè 0.7 + cần người duyệt Khi hai claim xung đột, thắng bên có độ tin cậy cao hơn; mọi quyết định ghi vào provenance (nguồn, độ tin cậy, vị trí đoạn trong file gốc). Nhờ đó UI có thể tô cảnh báo vàng ở đúng block, và đội ingest biết chính xác recognizer nào sai. interface Recognizer { id: string; priority: number; recognize(block: RawBlock, ctx: IngestContext): Claim | null; } // Thêm journal hay quy ước mới = thêm 1 recognizer hoặc 1 styleMap, không đụng code cũ (Open/Closed). 4.2 Những điểm kỹ thuật cần giải quyết trong ingest - Công thức: OMML → MathML (dùng XSLT chuẩn của Microsoft hoặc thư viện tương đương) → LaTeX; lưu cả omml gốc. Công thức không chuyển được thì status: 'fallback-image' (ảnh vector) và QC cảnh báo. Với MathType (OLE/WMF) coi là ảnh dự phòng cho tới khi có bộ chuyển. - Ảnh: đọc tách riêng thành asset; nhận diện đúng định dạng bằng magic bytes chứ không dựa đuôi; PNG/JPEG/BMP giữ nguyên; TIFF → PNG; EMF/WMF/EPS/PDF → SVG/PDF vector (chuyển đổi phía server khi cần). Tính DPI và kích thước để QC. - Bảng: dựng lưới trực tiếp từ gridSpan/vMerge, không đoán header từ chuỗi. Caption lấy từ style TableCaption (phía trên), TableNote và SourceLine (phía dưới). - Citation: nếu có field Zotero/EndNote thì lấy dữ liệu có cấu trúc làm Reference.csl; nếu không, parse [1], [2–4], [5,6] thành node cite có refIds. - Chạy trong Web Worker (Comlink) để không treo UI, và viết isomorphic để cùng package này chạy trong Node cho batch processing (xem mục 10). - .doc cũ: gửi lên DocConverterPort (LibreOffice phía server); nếu chưa có BE thì chặn từ UI với thông báo hướng dẫn lưu thành .docx. 5. Journal Profile, dàn trang và typography 5.1 Profile là dữ liệu, không phải code Hiện JOURNAL_TEMPLATES trong templates.ts chỉ có tên và màu, còn quy tắc khác nằm rải trong các Tab và latex-template.ts. Đề xuất mỗi tạp chí là một file JSON có schema (zod), có version, nạp qua JournalProfileProvider: interface JournalProfile { id: 'VNUHCM-HS' | string; version: string; names: { vi: string; en: string; short: string }; color: string; languages: ('vi' | 'en')[]; articleTypes: { id: string; label: Record<string, string>; requiredSections: string[] }[]; frontMatter: { abstract: { minWords: number; maxWords: number; structured: boolean }; keywords: { min: number; max: number; separator: ';' | ',' }; dateFormat: 'dd/MM/yyyy'; doiPattern: string; // để QC đối chiếu DOI với volume/issue }; citation: { style: 'vancouver'; inText: 'bracket' | 'superscript'; rangeDash: '–' }; styleMap: Record<string, string>; // style Word -> vai trò ngữ nghĩa (dùng ở ingest) layout: LayoutDefaults; // khổ giấy, cột, lề, header/footer, số trang typography: TypographyDefaults; // font, cỡ chữ, leading, tracking, giãn đoạn, hyphenation latex: { cls: string; engine: 'xelatex' | 'lualatex'; fonts: FontStack }; qc: { ruleset: string[]; weights: Record<string, number>; overrides?: object }; } Các mâu thuẫn ở mục 1 (tóm tắt 150–250 hay 275–500 từ, từ khóa phân cách, định dạng ngày, 4 hay 8 loại bài, [1] hay chỉ số trên) đều được*

giải quyết bằng cách đặt vào profile. Khi tạp chí đổi quy định, chỉ sửa một file. 5.2 Hợp nhất cấu hình lúc render resolved = deepMerge(globalDefaults, profile.layout/typography, manuscript.layoutOverrides) LayoutSettingsDrawer chỉ ghi vào layoutOverrides những gì khác profile, nên đổi tạp chí đích không làm mất tùy chỉnh hợp lệ và có thể bấm Đặt lại về mặc định của tạp chí. 5.3 Typography (đáp ứng nhóm yêu cầu Typography) Yêu cầu Nơi cấu hình Cách thực thi trong LaTeX Font, cỡ chữ, leading typography.fonts, size, leading fontspec, \linespread; có fallback font thật (danh sách ưu tiên), không còn thay chuỗi MinionPro→Times New Roman bằng string replace Tracking typography.tracking microtype (letterspace) Giãn đoạn paragraphSpacing, indent \parskip, \parindent Hyphenation typography.hyphenation { lang, leftMin, rightMin, enabled } polyglossia/babel theo Manuscript.lang; từng đoạn có thể mang lang riêng (tóm tắt tiếng Anh trong bài tiếng Việt). Tiếng Việt chủ yếu âm tiết đơn nên mặc định tắt; tiếng Anh bật Widow/orphan typography.widowOrphan \clubpenalty, \widowpenalty Lề, 1/2 cột, A4 layout.page geometry, multicol/twocolumn Header/footer, running title, số trang layout.header/footer/pageNumbering fancyhdr (chuyển logic hiện có pageNumbering sang đây) 5.4 Trang đầu và lớp .cls do bộ phận dàn trang sở hữu PDF mẫu ID630_HS cho thấy trang 1 là một bố cục đặc biệt: cột trái chứa Liên hệ/Lịch sử/DOI/Bản quyền/QR, khung Tóm tắt nền xanh nhạt, hộp Trích dẫn bài báo này ở chân trang, đầu trang lặp tên tạp chí và số trang. Những thứ này không nên sinh bằng TypeScript. Đề xuất: - Mỗi tạp chí có một lớp structpaper-hs.cls (hoặc .sty) do người làm TeX sở hữu, định nghĩa các macro như \spTitle, \spAuthors, \spAbstract, \spSidebar. - LaTeX emitter chỉ sinh lời gọi macro với nội dung đã escape. Đổi thiết kế trang = sửa .cls, không đụng TypeScript. - Số thứ tự (Figure, Table, Equation) do computeNumbering cấp, không dựa vào bộ đếm LaTeX. Hai bên khớp nhau khi preview và khi xuất. 6. Kiến trúc Editor 6.1 Một store, một nguồn sự thật Bỏ mô hình hai tầng formData ↔ PaperContext và isDirty set tay. Thay bằng một store (Zustand + Immer hoặc Redux Toolkit) giữ Manuscript. Hiệu năng gõ phím đạt được bằng selector theo ID và React.memo ở từng block, không phải bằng bản nháp cục bộ: - Component đoạn văn chỉ đăng ký selectBlock(id), nên gõ một đoạn không render lại cả bài. - Ô nhập giữ state cục bộ và commit vào store sau khoảng 150 ms (debounce). - Danh sách block dùng ảo hóa (react-virtuoso) vì bài tổng quan dài. - isDirty là giá trị suy ra: rev hiện tại khác rev đã lưu. 6.2 Command + History (Command, Memento) Mọi thay đổi đi qua CommandBus; không component nào gọi setState lên tài liệu trực tiếp. interface Command<P = unknown> { type: string; payload: P } // handler: (doc, payload) => { doc: Manuscript; patches: Patch[]; inverse: Patch[] } bus.dispatch({ type: 'block/insert', payload: { after: blockId, block } }); bus.dispatch({ type: 'reference/remove', payload: { id } }); // numbering tự cập nhật vì là dữ liệu dẫn xuất bus.dispatch({ type: 'table/mergeCells', payload: { id, range } }); Lợi ích: undo/redo miễn phí (dùng patch ngược của Immer), autosave chỉ cần lắng nghe luồng patch, op-log đẩy lên BE về sau, lịch sử revision là chuỗi patch, và mỗi command test được độc lập không cần React. Sửa tay isDirty, setPaper, updatePaper đều biến mất. 6.3 Feature module (cắt dọc) và Block registry Mỗi tab trong EditorTabNav là một feature package, có cùng khung: feature-references/ commands/ # reference/add, remove, reorder, parseRaw selectors/ # danh sách có số thứ tự, trạng thái cite ui/ # ReferencesTab, ReferenceItem qc/ # luật QC riêng của tính năng index.ts # export { tab, commands, qcRules } Với nội dung thân bài, dùng Block registry để một loại block là một đơn vị độc lập xuyên các tầng: registerBlock('table', { schema: TableBlockSchema, Editor: TableEditor, // feature-body/table toLatex: tableToLatex, // exporters/latex/blocks/table.ts toDocx: tableToDocx, toJats: tableToJats, qc: tableRules, }); TypeScript buộc kiểu Record<BlockType, Handler> ở mỗi tầng: thêm block mới (ví dụ equation) mà quên viết exporter sẽ lỗi biên dịch, nên không còn trường hợp sót. Ba người làm ba block khác nhau không đụng nhau. 6.4 Soạn thảo văn bản có cấu trúc Ô textarea hiện tại không thể giữ in nghiêng H. pylori, chỉ số dưới FEV_1, citation và cross-ref như những đối tượng thật. Đề xuất dùng TipTap/ProseMirror cho đoạn văn và tiêu đề: - Schema ProseMirror ánh xạ 1-1 với Inline[]; cite và xref là node có thể bấm, không phải regex tô màu. - Gõ [mở bộ chọn tài liệu tham khảo; chèn Figure/Table/Equation bằng danh sách mục tiêu. - Dán từ Word/Excel đi qua cùng bộ chuẩn hóa của ingest. - Công thức: preview bằng KaTeX, sửa bằng MathLive; bảng giữ modal dạng bảng tính hiện có, đọc/ghi lưới mới. - Chi phí: đây là phần tốn công nhất của Editor, nên đặt ở giai đoạn 4 (mục 14). Có thể làm sau khi store và command đã ổn. 6.5 Preview hai tầng Hiện chỉ có PDF bằng XeLaTeX nên mỗi lần đổi là chờ 1–2 giây và iframe nhấp nháy. Đề xuất: Tầng Nội dung Khi nào Preview nhanh React render từ AST theo profile, gần giống dàn trang Tức thì, trong lúc gõ Preview chính xác PDF từ compile service, hiển thị bằng pdf.js (giữ nguyên vị trí cuộn, đánh dấu lỗi trên trang) Debounce 1–2 s sau khi ngừng gõ hoặc bấm biên dịch Từ khi emitter chèn comment %%BLOCK:<id> vào TeX, có thể ánh xạ ngược lỗi LaTeX → block (bấm lỗi để nhảy tới đúng đoạn) và dùng SyncTeX cho bấm vào PDF → nhảy tới block. 6.6 Bus sự kiện cho các việc nền Các việc nền đăng ký lắng nghe luồng patch thay vì được gọi tay trong UnifiedEditorView: document patch ─┬─▶ autosave (debounce 1 s) ─▶ Repository.save ├─▶ QC engine (worker, debounce, chỉ tính lại block đổi) ─▶ qcStore ├─▶ compile scheduler (debounce, hủy job cũ) ─▶ previewStore └─▶ outbox (khi có BE) ─▶ đồng bộ UnifiedEditorView còn lại chỉ là khung bố cục: chia đôi màn hình, dock, tab. Không còn logic nghiệp vụ. 7. Quality Control / Preflight (rule engine) Hiện computePreflightCheck chấm 100 điểm, và docx-extractor.ts vừa trích xuất vừa chấm. Yêu cầu phần mềm đòi preflight rộng hơn nhiều (font missing, image resolution, overset text, lỗi link, lỗi numbering). Cách làm bền vững là mỗi luật là một đơn vị nhỏ (Specification pattern): interface Rule { id: string; // 'ref.uncited', 'fig.dpi', 'meta.doi-volume' severity: 'error' | 'warning' | 'info'; stage: 'static' | 'post-compile'; // post-compile đọc log/PDF appliesTo(profile: JournalProfile): boolean; run(ctx: QcContext): Issue[]; // ctx có doc, numbering, profile, compileReport? } interface Issue { ruleId; severity; message; target: { blockId?: Id; path?: string }; fix?: Command } - Profile chọn bộ luật qua qc.ruleset và trọng số điểm; tạp chí khác nhau có luật khác nhau mà không rẽ nhánh trong code. - fix là một Command: nút Sửa nhanh trong PreflightQCTab chỉ việc dispatch. - Chạy trong worker, debounce, memoize theo block để không tính lại cả bài. Nhóm Ví dụ luật Nguồn dữ liệu Metadata Thiếu tiêu đề/tác giả/email liên hệ; số từ khóa; độ dài tóm tắt theo profile; DOI không khớp volume/issue (lỗi có thật trong PDF mẫu) doc + profile Cấu trúc IMRAD theo articleType; mục bắt buộc thiếu (Ethics, Funding, Conflict, Data availability); style Guide* còn sót trong nội dung doc Numbering Figure/Table nhắc sau khi đã xuất hiện, hoặc không được nhắc; xref trỏ vào block không tồn tại; thứ tự đánh số khác thứ tự nhắc lần đầu computeNumbering Reference Cite không có reference; reference không được cite; trùng DOI; reference parse failed; thiếu năm/tập/trang doc Hình DPI thấp, kích thước quá nhỏ cho khổ cột, thiếu caption, thiếu dòng Nguồn AssetMeta Bảng Bảng là ảnh; thiếu caption hoặc dòng nguồn; bảng vượt quá chiều rộng doc Công thức Chuyển đổi thất bại (fallback-image); công thức thiếu số hoặc thiếu tham chiếu doc Sau biên dịch Overfull \hbox (overset text), font bị thay thế, Reference undefined, hình thiếu, font chưa nhúng, ảnh dưới DPI log TeX, pdffonts, pdfimages do compile worker trả về Báo cáo ingest (mục 4) và QC dùng chung kiểu Issue, nên giao diện chỉ có một danh sách vấn đề duy nhất. 8. Exporters và Compile service 8.1 Một AST, nhiều đầu ra (Strategy + Visitor) Tất cả đầu ra đọc cùng Manuscript đã qua computeNumbering và resolvedProfile: Exporter Đầu ra Ghi chú LatexExporter article.tex + references.bib Hợp nhất exportToLatex và generateFullJournalLatex. Dùng LaTeX writer nhỏ với hàm escapeTex bắt buộc cho mọi text, có golden test JatsExporter JATS XML 1.3 Đáp ứng yêu cầu Online publishing. Có sẵn nhờ cây section, xref, cite, Reference.csl và lang DocxExporter .docx Dùng lại bộ style của Template_EN/VI nên Word xuất ra khớp với đầu vào HtmlExporter HTML Dùng chung với preview nhanh ZipPackager tex + bib + images + (xml) Gói nộp tòa soạn hoặc Overleaf Mỗi exporter là một thư mục với mỗi block một file (blocks/table.ts, blocks/equation.ts…), thỏa kiểu Record<BlockType, Handler> (mục 6.3). 8.2 Compile

service tách riêng Loại bỏ việc biên dịch ở hai nơi. Next.js không còn route compile trong môi trường thật; compiler-worker là một container riêng, Spring Boot điều phối. Editor ──(manifest: tex + danh sách assetId)──▶ Spring /api/v1/compile ◀──────────── jobId, SSE trạng thái ──────────────┐ Spring ─ kiểm tra cache theo hash(tex + assets + profile) │ ─ asset thiếu thì client mới upload (HEAD theo sha256) ─ đẩy job vào hàng đợi ──▶ compiler-worker ──▶ PDF + log + báo cáo preflight - Không gửi lại base64 mỗi lần. Ảnh upload một lần theo hash; manifest chỉ liệt kê hash. Bỏ luôn hack alias fig1.png/fig1.jpg. - Cache theo hash nội dung (client giữ thêm một cache nhỏ ở IndexedDB), hủy job cũ khi có job mới của cùng bản thảo, debounce ở client. - Sandbox bắt buộc vì TeX do hệ thống sinh từ nội dung người dùng: chạy --no-shell-escape, openin_any=p, openout_any=p, container không có mạng, giới hạn CPU/RAM/thời gian, thư mục tạm riêng mỗi job, người dùng không root. Vẫn escape mọi text và chặn \input, \write18 do người dùng gõ vào (kể cả khi dán). - Cổng CompilerPort: dev dùng LocalCliCompiler (xelatex/tectonic), production dùng RemoteCompiler. UI không biết khác biệt. - Dây chuyền prepress cho PDF xuất bản: xelatex → Ghostscript (PDF/X hoặc PDF/A) → kiểm font nhúng (pdffonts) → kiểm DPI (pdfimages) → nhúng metadata (hyperxmp) → bookmarks. Kết quả preflight quay về QC (stage post-compile). - Rủi ro cần quyết sớm: PDF/X và PDF gắn thẻ truy cập (accessibility tagging) hỗ trợ tốt hơn trên LuaLaTeX so với XeLaTeX. Nếu truy cập là bắt buộc, nên làm thử LuaLaTeX trước khi dồn công vào .cls. Font thương mại (MinionPro, MyriadPro) cần giấy phép cho worker; nếu chưa có, profile phải khai báo font thay thế và QC hiện cảnh báo đang dùng font thay thế. 9. Persistence offline-first với IndexedDB (giảm tải BE), sẵn sàng cho Spring Boot + MySQL 9.1 Quyết định - IndexedDB là nơi làm việc chính của trình duyệt: mở bài, sửa, undo, preview đều không cần BE. Dùng Dexie (API gọn, migration theo version, hỗ trợ transaction và index). - Mọi truy cập đi qua ManuscriptRepository và AssetRepository (mục 2.2). Sau này BE chỉ là một adapter bổ sung và một decorator đồng bộ, không sửa Editor. - Bỏ sessionStorage: giới hạn khoảng 5 MB, mất khi đóng tab, và try {} catch {} hiện tại nuốt lỗi quota một cách im lặng. 9.2 Lược đồ Dexie Bảng Khóa / index Nội dung Ghi chú manuscripts id, index [journalId+updatedAt], status, syncState Manuscript JSON không chứa blob; rev, schemaVersion, title, updatedAt Danh sách Bản thảo của tôi chỉ cần cột nhẹ (title, updatedAt, status) để khỏi nạp cả bài revisions id, index [manuscriptId+rev] Chuỗi patch giữa các snapshot; snapshot đầy đủ mỗi N revision hoặc khi người dùng bấm Lưu mốc Đáp ứng yêu cầu Revision và proofing; có thể khôi phục assets hash (sha256) Blob, mime, size, width, height, dpi, vector Một ảnh dùng ở nhiều bản thảo chỉ lưu một lần assetRefs [manuscriptId+hash] Quan hệ bản thảo ↔ asset Dọn rác (GC) asset mồ côi ingestSources manuscriptId Blob .docx gốc + IngestReport Dùng cho Nội dung bóc tách, nhập lại khi có bản sửa (Revision) outbox ++seq, manuscriptId Thao tác chờ đẩy lên BE ({op, rev, patches}), attempts Chỉ dùng khi bật đồng bộ compileCache hash PDF Blob + thời gian Giới hạn LRU theo dung lượng settings key Profile đã chọn, cấu hình UI, consentAi Không lưu API key 9.3 Bảo đảm độ bền - Autosave debounce khoảng 1 giây từ luồng patch; ghi manuscripts và revisions trong cùng một transaction để không lệch. - Gọi navigator.storage.persist() và theo dõi navigator.storage.estimate(). Khi gần hết quota, hiển thị cảnh báo và đề nghị xuất ZIP; bắt QuotaExceededError thật, không nuốt lỗi. - Safari/ITP có thể xóa dữ liệu website sau thời gian không dùng, nên IndexedDB không được coi là kho lưu trữ duy nhất khi bài quan trọng. Đây chính là lý do cần BE đồng bộ, và trước đó cần nút Xuất bản sao lưu (.structpaper = zip gồm JSON và asset). - Nhiều tab: dùng Web Locks API hoặc BroadcastChannel để một bản thảo chỉ được sửa ở một tab; tab khác mở chế độ chỉ đọc. - Migration: mỗi lần đổi lược đồ thì tăng db.version(n) và có hàm upgrade; riêng Manuscript có schemaVersion và chuỗi migrate(vN→vN+1) có test với fixture. 9.4 Chiến lược đồng bộ khi có BE SyncedRepository là decorator quanh IndexedDbRepository: ghi cục bộ trước, đưa vào outbox, đẩy khi có mạng. Giai đoạn Cơ chế Khi nào dùng S1 Đồng bộ theo bản sao: mỗi bản thảo có rev; lưu lên BE bằng If-Match: <rev> (khóa lạc quan). Xung đột thì giữ cả hai (tạo bản xung đột) và để người dùng chọn Tác giả làm một mình hoặc luân phiên; đủ cho giai đoạn đầu S2 Đồng bộ theo chuỗi patch (op-log) đã có sẵn từ Command Khi cần giảm băng thông và lưu lịch sử chi tiết trên BE S3 CRDT (Yjs gắn với ProseMirror) Chỉ khi thật sự cần nhiều người sửa đồng thời Khuyến nghị bắt đầu từ S1. Vì mọi thay đổi đã là Command nên nâng lên S2 không phải viết lại Editor. 10. Backend Spring Boot + MySQL 10.1 Cấu trúc (modular monolith, tách service khi thật cần) vn.edu.vnu.structpaper identity/ # đăng nhập (JWT/OIDC), vai trò: Author, Editor, Production, Admin manuscript/ # bản thảo, revision, quyền truy cập asset/ # upload theo sha256, metadata ảnh, lưu S3/MinIO hoặc filesystem journal/ # JournalProfile (version), template .cls compile/ # job, hàng đợi, kết quả, cache ingest/ # chuyển .doc → .docx (LibreOffice), batch ingest bằng Node worker ai/ # proxy OpenAI, ghi log sử dụng common/ # lỗi chuẩn, audit, cấu hình Mỗi gói có controller, service, repository, DTO riêng; không gọi chéo vào repository của gói khác (qua service/interface). Kiểm bằng ArchUnit. 10.2 Lược đồ MySQL (rút gọn) Bảng Cột chính Ghi chú users id, email, name, role journals / journal_profiles journal_id, version, profile_json (JSON), active Profile có version; bản thảo ghi nhớ version đã dùng manuscripts id (ULID), owner_id, journal_id, title, doi, status, rev, schema_version, doc_json (JSON), updated_at @Version cho khóa lạc quan; cột phụ (title, doi, status) để tìm kiếm mà không cần mở JSON manuscript_revisions manuscript_id, rev, kind (patch/snapshot), payload, author_id, created_at Khớp revisions ở IndexedDB assets sha256 (PK), size, mime, width, height, dpi, storage_key manuscript_assets manuscript_id, sha256 compile_jobs id, manuscript_id, rev, tex_hash, status, started_at, finished_at, pdf_key, log_key, report_json batches id, owner_id, status, total, done Batch processing audit_log actor, action, target, at Manuscript là một aggregate nên lưu dạng JSON trong một dòng (MySQL 8 JSON), đồng thời phi chuẩn hóa vài cột để truy vấn. Dùng Flyway cho migration. 10.3 Hợp đồng FE–BE - OpenAPI là nguồn sự thật. Sinh client TypeScript (openapi-typescript hoặc Orval) và DTO Java từ cùng một file, để hai bên không lệch. - Manuscript được kiểm bằng JSON Schema sinh từ zod; BE (thư viện json-schema-validator) từ chối tài liệu sai schemaVersion. Contract test chạy trong CI cả hai phía. - Endpoint chính: Endpoint Việc GET/PUT /api/v1/manuscripts/{id} Đọc và ghi bản thảo (If-Match) GET /api/v1/manuscripts/{id}/revisions Lịch sử HEAD/PUT /api/v1/assets/{sha256} Kiểm tra và upload ảnh theo hash POST /api/v1/compile rồi GET /api/v1/compile/{jobId} (hoặc SSE) Biên dịch GET /api/v1/journals/{id}/profile Profile có version POST /api/v1/ingest/convert .doc → .docx POST /api/v1/batches Nạp zip nhiều .docx, xử lý hàng loạt POST /api/v1/ai/extract Proxy AI 10.4 Batch/API/workflow (yêu cầu Automation) Nhờ ingest-docx isomorphic, chạy cùng package trong một Node worker nối vào hàng đợi: POST /batches nhận zip, mỗi .docx thành một job (ingest → QC → compile), trả báo cáo tổng hợp. Không viết parser lần thứ hai bằng Java. 11. AI: trợ lý đề xuất, không ghi đè - Khóa API chỉ ở BE. Bỏ NEXT_PUBLIC_OPENAI_API_KEY và ô nhập sk-… ở client. Client gọi AiPort → POST /api/v1/ai/extract. - Chỉ gửi phần cần thiết: front-matter và các block mà recognizer đánh giá mơ hồ, không gửi cả bản thảo. Giảm token, giảm rủi ro lộ bản thảo chưa công bố. Có công tắc đồng ý gửi nội dung ra dịch vụ AI (consentAi) và ghi log sử dụng. - Kết quả là đề xuất dạng Command kèm độ tin cậy; giao diện hiển thị khác biệt để người dùng chấp nhận hoặc từ chối từng thay đổi. Không còn "merge đè lên dữ liệu OpenXML". - Dùng đầu ra có cấu trúc (JSON Schema) và prompt có version trong ai-client; test bằng bộ fixture, mô hình giả (MockAi) cho CI. - Tên mô hình nằm trong cấu hình BE; UI không hard-code danh sách model. 12. Bảng design pattern (dùng ở đâu, để làm gì) Pattern Áp dụng Giải quyết vấn đề Hexagonal (Ports & Adapters) Repository, Compiler, AI, Converter, Profile Đổi IndexedDB ↔ Spring, local ↔ remote mà không sửa Editor Anti-corruption layer ingest-docx: kiểu OOXML không lọt ra ngoài package Giữ lõi miền sạch Pipeline / Chain of Responsibility Các tầng ingest, bộ Normalizer Mỗi tầng một người làm, thêm bước không sửa bước cũ Strategy Recognizer, Exporter, Compiler, chính sách Track Changes Thay thuật toán theo ngữ cảnh Visitor Duyệt AST: numbering, export, QC Một cách

duyệt, nhiều xử lý Registry / Plugin Block registry, Journal Profile Thêm block hoặc tạp chí không chạm code cũ (Open/Closed) Command + Memento Mọi thao tác Editor, undo/redo, revision Nhật ký thay đổi duy nhất Repository + Decorator SyncedRepository bọc IndexedDbRepository Thêm đồng bộ mà không đổi interface Observer / Event bus Autosave, QC, compile, outbox Bỏ logic điều phối khỏi UnifiedEditorView Specification Luật QC Luật nhỏ, ghép được, bật tắt theo profile Builder Assembler (ingest), LaTeX writer Dựng cấu trúc phức tạp từng bước Factory createEmptyManuscript(profile), tạo block Thay createEmptyPaper và các hàm rải rác Adapter OpenAI, TeX engine, LibreOffice Che chi tiết bên ngoài Derived data (selector) computeNumbering Số thứ tự không bao giờ lệch Không dùng hết cùng lúc ở ngày đầu. Ba pattern bắt buộc có sớm là Command, Repository và Registry (cùng Derived data); phần còn lại thêm dần theo lộ trình. 13. Chiến lược kiểm thử và chất lượng mã Loại Cách làm Ví dụ Golden ingest fixtures/.docx → so sánh JSON kỳ vọng (snapshot) Template_EN, Template_VI, 8 bài mẫu hiện có; mỗi lỗi parser sau này thêm một fixture Unit theo tầng Đoạn XML nhỏ cho từng assembler/normalizer VietnameseSpacing: Nguy ễ n → Nguyễn; NFD→NFC Property-based fast-check cho computeNumbering Chèn/xóa ngẫu nhiên block rồi kiểm số liên tục, không trùng, cite khớp Golden export LaTeX/JATS/DOCX so với file mẫu Có escapeTex cho ký tự đặc biệt, tiếng Việt Visual regression Biên dịch bài mẫu, pdftoppm + pixelmatch với PDF tham chiếu (ID630_HS) Bắt lệch layout khi sửa .cls Contract OpenAPI + JSON Schema chạy ở cả FE và BE Sai schemaVersion bị từ chối E2E Playwright: upload → sửa → autosave → F5 → khôi phục → xuất PDF Kiểm đúng lỗi mất dữ liệu hiện nay Kiến trúc dependency-cruiser, ArchUnit Vi phạm tầng thì fail CI Quy ước chung: TypeScript strict, zod ở mọi biên (nhập file, đọc IndexedDB, gọi API), lỗi trả về dạng Result/Issue thay vì catch {} im lặng, mỗi package có README ghi trách nhiệm và API công khai. 14. Lộ trình chuyển đổi (strangler, không viết lại từ đầu) Mục tiêu: sản phẩm luôn chạy được ở cuối mỗi giai đoạn. Giai đoạn Việc chính Kết quả kiểm chứng 0. Đóng băng hành vi Dựng monorepo, CI, dependency-cruiser. Viết characterization test cho parser và exporter hiện tại bằng 8 bài mẫu và 2 template Biết chắc đang chạy thế nào trước khi đổi 1. Lõi doc-model Manuscript v2, zod, computeNumbering, migrateV1toV2 + adapter ngược cho UI cũ Số thứ tự và cite/xref tính đúng; property test xanh 2. Lưu trữ storage với Dexie (manuscripts, assets, revisions), autosave, trang Bản thảo của tôi, nút sao lưu .structpaper. Tách base64 thành asset F5/đóng tab không mất dữ liệu; bài nhiều ảnh lớn vẫn lưu được 3. Ingest pipeline Chạy song song parser cũ và mới, so kết quả. Làm StyleMapRecognizer trước (ăn ngay Template_EN/VI), sau đó assemblers, OMML→MathML→LaTeX, ảnh/magic bytes. Chuyển vào Web Worker Golden test xanh; parser cũ được gỡ khi mới đạt ngang 4. Editor core Store + CommandBus + history; chuyển từng tab sang feature module; Block registry; thay textarea bằng ProseMirror cho đoạn văn Undo/redo; UnifiedEditorView không còn logic nghiệp vụ 5. QC + Exporters Rule engine, gộp hai bộ sinh LaTeX thành một, thêm exporter JATS, tách .cls cho HS Báo cáo QC thống nhất; XML JATS hợp lệ 6. Backend Spring Boot (manuscript, asset, journal, identity), SyncedRepository S1, compiler-worker có sandbox và cache, AI proxy Mở bài ở máy khác; compile an toàn 7. Production PDF/X + prepress, post-compile preflight, proofing (comment trên PDF), batch, S2 (đồng bộ theo patch) Đáp ứng đầy đủ file yêu cầu Giai đoạn 3 và 4 có thể làm song song nhờ doc-model đã chốt ở giai đoạn 1. 15. Chia việc theo team (hết cảnh "dồn cục") Nhóm Sở hữu (CODEOWNERS) Phụ thuộc Model & Core doc-model, journal-profiles (schema) không Ingest ingest-docx (mỗi assembler/recognizer một người, ví dụ Bảng, Hình, Công thức, Reference) doc-model Editor editor-core, feature-* doc-model, storage, qc-engine Xuất bản exporters, compiler-worker, .cls cho từng tạp chí doc-model QC qc-engine và luật doc-model Backend apps/api, contracts/ OpenAPI Hạ tầng CI, Docker, observability tất cả Như vậy việc "Bảng", "Hình", "Equation", "Reference", "Hyphenation" trong file yêu cầu là các thư mục và người sở hữu khác nhau: ingest/assemblers/table.ts, exporters/latex/blocks/table.ts, feature-body/table/, qc-engine/rules/table.ts, và typography nằm trong profile cùng .cls. Một thay đổi ở bảng không buộc ai đọc 2.100 dòng của parser. 16. Ma trận phủ yêu cầu phần mềm Yêu cầu (file yêu cầu) Nơi đáp ứng Input DOCX/DOC, Track Changes ingest-docx (chính sách Track Changes, .doc qua DocConverterPort) Nhận diện Heading, tác giả, tóm tắt, từ khóa, reference… StyleMapRecognizer + StructureRecognizer + assemblers Template riêng từng journal, nhiều loại bài JournalProfile (articleTypes, styleMap, layout) Layout 1/2 cột, A4, header/footer, số trang profile.layout + layoutOverrides + .cls Typography (font, leading, tracking, hyphenation) profile.typography → fontspec, microtype, polyglossia Hình: nhiều định dạng, vector, multi-panel, kiểm DPI Asset store + figure.panels + QC fig.* Bảng: nhiều dòng/cột, ngang, qua trang, lặp header Lưới rowSpan/colSpan + longtable/sidewaystable + Table N (continued) Công thức: OMML/MathType/LaTeX, đánh số, cross-ref Block equation + OMML→MathML→LaTeX + xref References Vancouver, renumber, link citation Reference.csl, cite theo ID, computeNumbering Cross-reference tự cập nhật Node xref theo ID DOI/ORCID/metadata FrontMatter + QC + hyperxmp trong PDF PDF publication/prepress Compile worker + Ghostscript/PDF/X + preflight sau biên dịch XML/JATS JatsExporter Proofreading PDF Bản compile có version + comment (giai đoạn 7) Revision không dàn lại từ đầu revisions + ingestSources + nhập lại theo provenance Batch/API/workflow Node worker dùng lại ingest-docx, POST /batches Quality control qc-engine 17. Các điểm cần bạn xác nhận 1. Cộng tác thời gian thực có trong phạm vi không? Nếu không, dừng ở đồng bộ S1/S2 và bỏ CRDT. 2. PDF/X và PDF gắn thẻ truy cập có bắt buộc không? Nếu có, nên thử LuaLaTeX sớm, trước khi viết .cls. 3. Font thương mại (MinionPro, MyriadPro) có giấy phép dùng trên máy chủ không? Nếu không, cần chọn font thay thế chính thức. 4. Trích dẫn trong bài: theo template là [1], còn PDF mẫu là chỉ số trên. Mặc định cho từng tạp chí là gì? 5. Mức tin cậy với template: có bắt buộc tác giả dùng Template_EN/VI chuẩn (ingest theo style, gần như chính xác tuyệt đối) hay vẫn phải chấp nhận Word tự do (cần heuristic/AI nhiều hơn)? Sau khi bạn chốt các điểm trên mình có thể viết tiếp đặc tả chi tiết cho doc-model (zod schema đầy đủ), StyleMapRecognizer cho Template_VI/EN, và lược đồ Dexie dưới dạng mã. trên đây là đề xuất cho strucpaper của tôi nè, tôi cần hỏi bạn một số câu hỏi*

Response: Bản đề xuất kiến trúc này rất chặt chẽ, tư duy phân tách ranh giới hệ thống (Bounded Context / Hexagonal) và xử lý bài toán Document Modeling chuẩn chỉnh. Việc biến **Manuscript thành AST có ID ổn định**, biến **số thứ tự/citation thành derived data** và **Journal Profile thành configuration** đã giải quyết tận gốc các điểm nghẽn nghiêm trọng của bản v1.

Bạn có những câu hỏi hay băn khoăn cụ thể nào cần mổ xẻ thêm? Ví dụ:

- Chi tiết thiết kế Zod schema / AST node cho `doc-model`?
- Pipeline chuyển đổi OMML → MathML → LaTeX và xử lý MathType/vector asset?
- Chiến lược đồng bộ State, Undo/Redo với Immer Patch trong `editor-core`?
- Cơ chế sandbox và pipeline tiền/hậu biên dịch (PDF/X, Ghostscript, XeLaTeX vs LuaLaTeX) trên Spring Boot & Worker?
- Hoặc giải quyết 5 điểm cần chốt ở mục 17?

Bạn cứ nêu các câu hỏi nhé!

User prompt: Việc biến Manuscript thành AST có ID ổn định, biến số thứ tự/citation thành derived data và Journal Profile thành configuration đã giải quyết tận gốc các điểm nghẽn nghiêm trọng của bản v1. ý này là sao vậy, trong này dùng nhiều thuật ngữ tiếng anh quá

Response: Ý này thực chất chỉ ra **3 giải pháp cốt lõi** để sửa các lỗi ngớ ngẩn mà bản v1 đang gặp phải.

Dưới đây là giải thích bình dân, đi thẳng vào nguyên nhân bản cũ bị lỗi và cách bản mới xử lý:

1. Manuscript thành AST có ID ổn định (Thay vì một chuỗi chữ dài ngoằng)

- **Bản v1 bị gì?** Bản v1 lưu bài báo như một chuỗi văn bản phẳng hoặc các ô chữ thô (`content: string`). Khi bạn muốn in nghiêng một chữ như *H. pylori*, gắn chỉ số dưới như FEV_1, hay chèn một công thức toán, hệ thống không hiểu từng chữ cái đó là gì, rất dễ làm mất định dạng hoặc vỡ cấu trúc khi xuất sang LaTeX.
- **AST có ID ổn định là gì?**
  - **AST (Abstract Syntax Tree - Cây cú pháp trừu tượng):** Thay vì một chuỗi chữ, tài liệu được xẻ nhỏ thành một cái cây có phân cấp (Bài báo $\rightarrow$ Chương $\rightarrow$ Đoạn văn $\rightarrow$ Từng mẩu chữ nhỏ). Mỗi mẩu chữ nhỏ sẽ mang nhãn riêng: chữ này in đậm, chữ kia là link, chữ nọ là công thức toán.
  - **ID ổn định (Stable ID):** Mỗi đoạn văn, mỗi bức ảnh, mỗi bảng biểu đều được cấp một "chứng minh nhân dân" (mã định danh duy nhất, ví dụ `fig-01`, `sec-intro`).
- **Lợi ích:** Khi bạn kéo thả đổi chỗ đoạn văn, sửa nội dung hay tham chiếu tới một bức ảnh, hệ thống chỉ cần nhìn vào cái "mã định danh" đó để tìm đúng đối tượng, không sợ bị nhầm hay mất dấu.

2. Số thứ tự / citation thành derived data (Dữ liệu dẫn xuất, không gõ chết)

- **Bản v1 bị gì?** Bản cũ lưu cứng số thứ tự vào bài viết: gõ thẳng chữ *"Hình 1"*, *"Bảng 2"*, hoặc trích dẫn tài liệu `[1]`, `[2]`. Nếu bạn chèn thêm một bức hình mới vào ngay đầu bài, toàn bộ số phía sau phải sửa tay lại: Hình 1 thành Hình 2, Hình 2 thành Hình 3... Nếu lỡ tay xóa tài liệu tham khảo số `[3]`, trích dẫn trong bài sẽ bị lộn xộn hoặc chỉ vào hư không.
- **Derived data (Dữ liệu dẫn xuất) là gì?** Là **dữ liệu tự động tính toán ra từ nguồn khác chứ không lưu chết**.
  - Trong bài viết, bạn chỉ lưu: *"Xem hình {id: fig-so-do-he-thong}"* và trích dẫn *"{cite: tai-lieu-cua-ong-a}"*.
  - Một hàm chạy ngầm (`computeNumbering`) sẽ tự đếm: bức hình nào xuất hiện trước thì gán nhãn là "Hình 1", tài liệu nào được nhắc tới đầu tiên thì tự mang số "[1]".
- **Lợi ích:** Bạn chèn thêm, xóa bớt hay đảo lộn vị trí các bảng biểu, tài liệu tham khảo thoải mái; toàn bộ số thứ tự trong toàn bài và danh mục tài liệu tham khảo sẽ **tự động nhảy lại chính xác 100%**, không bao giờ sợ lệch.

3. Journal Profile thành configuration (Dữ liệu cấu hình, không viết cứng vào code)

- **Bản v1 bị gì?** Quy tắc của các tạp chí bị viết cứng (hard-code) rải rác khắp nơi trong mã nguồn: chỗ này viết code `if (tạp chí == A) thì tóm tắt tối đa 250 từ`, chỗ kia lại viết `if (tạp chí == B) thì trích dẫn dùng dấu ngoặc vuông`. Khi tạp chí đổi quy định hoặc muốn thêm một tạp chí mới, lập trình viên phải lục tung hàng nghìn dòng code ra để sửa, rất dễ sinh lỗi.
- **Configuration (Cấu hình) là gì?** Toàn bộ luật chơi của một tạp chí được tách riêng ra thành **một file dữ liệu độc lập** (file JSON):
  - Tạp chí này yêu cầu tóm tắt bao nhiêu từ?
  - Ngày tháng hiển thị dạng ngày/tháng/năm hay năm-tháng-ngày?
  - Font chữ gì, lề trang bao nhiêu, trích dẫn kiểu gì?
- **Lợi ích:** Muốn đổi quy định của tạp chí hoặc thêm tạp chí thứ 3, thứ 4, bạn chỉ cần tạo hoặc chỉnh sửa một file cài đặt (file cấu hình) mà **không cần đụng tới một dòng code tính năng nào của hệ thống**.

User prompt: Tách lõi miền (doc-model) thuần TypeScript, không dính React, DOM, IndexedDB hay HTTP. ý này là sao vậy

Response: Ý này có nghĩa là: **Rút toàn bộ định nghĩa về cấu trúc bài báo và các quy tắc tính toán của nó ra thành một thư viện độc lập, hoàn toàn "mù" về giao diện, trình duyệt và mạng.**

Nói một cách dân dã: **Tách biệt tuyệt đối giữa "Nội dung cuốn sách" và "Giá sách hay người đọc".**

1. "Không dính" những thứ kia cụ thể là gì?

- **Không dính React:** Không chứa thẻ JSX (`<div>`, `<p>`), không dùng Hook (`useState`, `useEffect`), không quan tâm giao diện hiển thị màu gì hay nút bấm nằm ở đâu.
- **Không dính DOM:** Không dùng các lệnh thao tác trực tiếp trên trình duyệt như `document.getElementById`, `window`, hay parse XML bằng `DOMParser`.
- **Không dính IndexedDB / HTTP:** Không chứa code lưu dữ liệu vào ổ cứng trình duyệt (`IndexedDB`, `Dexie`) và không chứa code gọi mạng (`fetch`, `axios`).

2. Vậy bên trong `doc-model` chứa cái gì?

Nó chỉ là mã nguồn **TypeScript nguyên bản (Pure TypeScript)** làm 3 nhiệm vụ:

1. **Định nghĩa hình hài bài báo (Types & Schema):**
   - Một `Manuscript` gồm những trường nào (tiêu đề, tác giả, danh sách block).
   - Một đoạn văn (`Paragraph`), cái bảng (`Table`), công thức (`Equation`) có cấu trúc ra sao.
   - Dùng thư viện kiểm tra dữ liệu như `Zod` để đảm bảo dữ liệu hợp lệ.
2. **Các hàm toán học / thuật toán thuần túy (Pure Functions):**
   - Hàm đếm lại số thứ tự: `computeNumbering(doc)` (nhận vào bài báo, trả về danh sách số Hình 1, Bảng 2).
   - Hàm chuyển đổi phiên bản dữ liệu: `migrateV1toV2(oldDoc)`.
3. **Các quy tắc nghiệp vụ:**
   - Những luật kiểm tra logic nội dung mà không cần quan tâm đến môi trường chạy.

3. Tại sao phải làm như vậy? (Lợi ích thực tế)

Vấn đề trước đây (Dính chùm)	Khi tách `doc-model` thuần TypeScript
Không chạy được ở nhiều nơi: Code viết dính `window` hay DOM thì chỉ chạy được trên trình duyệt; mang sang chạy batch trên NodeJS hoặc Web Worker sẽ bị văng lỗi ngay.	**Chạy được khắp mọi nơi (Isomorphic):** Cùng một file code đó, chạy trên giao diện web (React), chạy ngầm dưới Web Worker, chạy trên server (NodeJS) làm xử lý hàng loạt đều trơn tru vì nó không đòi hỏi môi trường đặc thù.
Cực kỳ khó viết kiểm thử (Test): Muốn test xem hàm đánh số Hình/Bảng có đúng không lại phải giả lập môi trường trình duyệt, dựng giao diện React rất cồng kềnh.	**Kiểm thử siêu nhanh và dễ:** Viết Unit Test chỉ mất vài mili-giây: đưa vào một đoạn dữ liệu JSON giả → gọi hàm → kiểm tra kết quả trả về, không cần bật trình duyệt.
Dễ hỏng dây chuyền: Sửa giao diện hoặc đổi thư viện lưu trữ cơ sở dữ liệu có thể vô tình làm vỡ logic tính số thứ tự bài báo.	**Lõi bất biến:** Cho dù ngày mai bạn đập bỏ React để chuyển sang Vue, hay bỏ IndexedDB để chuyển sang lưu thẳng lên MySQL, toàn bộ phần định nghĩa bài báo và thuật toán xử lý logic vẫn giữ nguyên 100%, không phải sửa một dòng.

User prompt: Manuscript v2: cây block + inline node + ID ổn định; số Figure/Table/Equation/Reference là dữ liệu dẫn xuất, không lưu cứng manuscript là cái gì vậy

Response: **Manuscript** dịch sát nghĩa sang tiếng Việt là **"Bản thảo"** (ở đây chính là **toàn bộ nội dung một bài báo khoa học** mà tác giả tải lên hệ thống để biên tập và xuất bản).

Trong hệ thống StructPaper:

- **Manuscript v1 (Bản cũ):** Lưu bài báo theo kiểu một túi dữ liệu phẳng (một vài ô nhập text thô, ảnh dán trực tiếp base64 vào, ai gõ gì lưu nấy).
- **Manuscript v2 (Bản mới):** Biến toàn bộ bài báo thành một cấu trúc phân tầng rõ ràng gồm hai cấp độ: **Block** (khối lớn) và **Inline** (phần tử nhỏ bên trong dòng).

Cụ thể từng phần trong câu đó có nghĩa là gì?

1. Cây block (Block Tree)

Bài báo được chia thành các khối độc lập xếp từ trên xuống dưới, giống như các khối Lego:

- **Khối tiêu đề** (Heading)
- **Khối đoạn văn** (Paragraph)
- **Khối bảng biểu** (Table)
- **Khối hình ảnh** (Figure)
- **Khối công thức toán** (Equation)

Mỗi khối này đứng riêng một hàng và mang đầy đủ thuộc tính của nó, không bị dính chùm vào nhau.

2. Inline node (Phần tử nội dòng)

Bên trong một khối (ví dụ bên trong một đoạn văn), chữ không phải là một chuỗi thô liên tục mà được chẻ nhỏ thành các "hạt" (node):

- Hạt chữ thường: `"Nghiên cứu về vi khuẩn "`
- Hạt chữ in nghiêng: `*H. pylori*`
- Hạt chữ chỉ số dưới: `FEV₁`
- Hạt công thức nằm trong dòng: `$x^2 + y^2 = z^2$`
- Hạt trích dẫn: nút trỏ tới tài liệu tham khảo (không phải gõ chữ `[1]` mà là một liên kết thông minh).

Nhờ có cấu trúc hạt này, việc xuất sang LaTeX hay HTML không bao giờ bị mất in nghiêng, chỉ số hay vỡ ký tự đặc biệt.

3. ID ổn định (Stable ID)

Mỗi khối (Block) và mỗi phần tử (Inline) khi được tạo ra đều được gắn một mã định danh duy nhất (ID) không đổi, ví dụ:

- Khối hình ảnh mang ID: `fig-abc123`
- Khối công thức mang ID: `eq-xyz789`
- Tài liệu tham khảo mang ID: `ref-hoang2024`

Cho dù bạn kéo khối hình ảnh từ trang 1 xuống trang 5, hay sửa đổi nội dung xung quanh nó, mã `fig-abc123` vẫn giữ nguyên.

4. Số Figure/Table/Equation/Reference là dữ liệu dẫn xuất, không lưu cứng

- **Lưu cứng (Bản cũ):** Trong bài viết lưu chết chữ *"Hình 1"*, *"Bảng 2"*, *"Công thức (3)"*, trích dẫn `"[1]"`. Khi bạn chèn thêm một hình mới vào đầu bài, toàn bộ số thứ tự phía sau bị sai hết và phải đi sửa tay từng số.
- **Dữ liệu dẫn xuất (Bản mới):** Trong bài viết không lưu số nào cả. Trong đoạn văn chỉ lưu: *"Xem hình {fig-abc123}"*.
    - Khi hiển thị ra màn hình hoặc xuất PDF, một thuật toán sẽ tự động đếm từ trên xuống dưới: hình nào gặp trước thì tự dán nhãn là "Hình 1", hình nào gặp sau thì tự là "Hình 2".
    - Thêm, bớt, xóa, đảo vị trí bất kỳ đâu thì toàn bộ số thứ tự tự động nhảy lại tức thì và chính xác tuyệt đối.

User prompt: khi mà mình đổi cách lưu như vậy thì lúc tạo rà file latex tiến hành biên dịch cũng phải tạo theo một kiểu mới luôn phải không, tài liệu có đề cập đến việc này không

Response: Có, tài liệu **đề cập rất rõ và dành hẳn mục 5.4 và mục 8.1** để giải quyết việc sinh ra file LaTeX theo cách mới.

Khi chuyển sang mô hình mới (Manuscript v2), cách tạo file LaTeX thay đổi hoàn toàn so với bản v1 ở 4 điểm mấu chốt sau:

1. Thay vì "cộng chuỗi thủ công", chuyển sang "duyệt cây theo từng loại khối" (Visitor Pattern)

- **Bản cũ (v1):** Có tới **2 file sinh LaTeX độc lập** (`exporters.ts` và `latex-template.ts`) dùng các lệnh nối chuỗi text hoặc `replace()`. Hai file này dễ bị lệch nhau, sửa chỗ này thì quên chỗ kia.
- **Bản mới (v2):** Gộp thành **một bộ sinh LaTeX duy nhất** (`LatexExporter`). Do bài báo lúc này là một cái cây gồm các khối (Block), hệ thống sẽ duyệt qua từng khối và gọi hàm xử lý tương ứng:
  - Gặp khối bảng → gọi `tableToLatex` (xuất ra `\begin{table}...`)
  - Gặp khối công thức → gọi `equationToLatex` (xuất ra `\begin{equation}...`)
  - Gặp đoạn văn → duyệt qua từng "hạt chữ" (inline), chữ nghiêng thành `\textit{}`, công thức nhỏ thành `$x^2$`, trích dẫn thành `\cite{...}`.
  - Mọi ký tự đặc biệt (`%`, `_`, `&`, `#`...) đều bắt buộc đi qua hàm chuẩn hóa `escapeTex` để không làm sập trình biên dịch LaTeX.

2. Số thứ tự (Hình, Bảng, Công thức) do hệ thống cấp trực tiếp, không để LaTeX tự đếm

- **Bản cũ:** Thường phụ thuộc vào bộ đếm ngầm của LaTeX (`\ref`, `\label`) hoặc gõ chết chuỗi `"Hình 1"`.
- **Bản mới:** Hàm tính toán `computeNumbering` đã xác định chính xác ảnh nào là Hình 1, bảng nào là Bảng 2 ngay từ đầu. Khi sinh file `.tex`, hệ thống gắn trực tiếp con số này vào nhãn để:
  - Số thứ tự trên màn hình soạn thảo (Editor), khi xem trước (Preview) và khi xuất ra PDF thành phẩm **luôn khớp nhau 100%**.

3. Tách giao diện trang bìa vào file khuôn mẫu riêng (`.cls`) thay vì nhồi code vào TypeScript

- **Tài liệu phát hiện:** Trang đầu của các tạp chí (như mẫu ID630_HS) có bố cục rất phức tạp: cột bên trái chứa thông tin liên hệ, DOI, bản quyền, mã QR; khung Tóm tắt nền xanh; hộp trích dẫn chân trang...
- **Giải pháp mới:**
  - Người chuyên về LaTeX sẽ tạo một file khuôn mẫu riêng (ví dụ `structpaper-hs.cls` do đội layout phụ trách). Trong đó định nghĩa sẵn các lệnh rút gọn như `\spTitle{...}`, `\spAuthors{...}`, `\spAbstract{...}`.
  - Bộ sinh LaTeX bên TypeScript chỉ việc điền nội dung vào các lệnh đó rồi gọi ra. Khi tạp chí đổi kiểu trang bìa, người làm TeX chỉ cần sửa file `.cls`, **lập trình viên không phải đụng vào code TypeScript**.

4. Gắn nhãn ngược để bấm vào lỗi là nhảy đến đúng đoạn (`SyncTeX` & `%%BLOCK:<id>`)

- Tài liệu đề xuất khi sinh file LaTeX, bộ emitter sẽ chèn thêm các dòng ghi chú ẩn dạng `%%BLOCK:<id>` tương ứng với ID của từng khối.
- Nhờ vậy, khi chạy lệnh biên dịch LaTeX:
  - Nếu dòng lệnh bị lỗi cú pháp → hệ thống biết chính xác lỗi nằm ở khối nào để đánh dấu đỏ ngay trên giao diện soạn thảo.
  - Hỗ trợ tính năng: Bấm vào một vị trí trên file PDF xem trước → màn hình tự cuộn đến đúng đoạn văn đó trong editor.

User prompt: cái ý số 3 á thì thực ra là nguyên cái template là đã có code rồi mà, đâu chỉ riêng trang bìa, sao các phần khác ta cũng ko làm như vậy luôn, chỉ cần tiêm nội dung vào template sẵn có thôi

Response: Ý tưởng của bạn — **"coi cả bài báo như một cái khuôn (template) có sẵn, chỗ nào có nội dung thì đục lỗ rồi tiêm (inject) chữ vào"** — là cách tiếp cận trực giác nhất. Thậm chí nhiều hệ thống đơn giản ngoài thực tế cũng từng làm như vậy bằng các bộ template engine (như Jinja, Mustache, Freemarker, Handlebars).

Tuy nhiên, trong một tài liệu khoa học học thuật phức tạp, **phần Front Matter (trang bìa) và phần Body (thân bài) có bản chất toán học/cấu trúc hoàn toàn khác nhau**, nên không thể áp dụng chung cách "đục lỗ tiêm nội dung".

Sự khác biệt cốt lõi nằm ở 4 lý do kỹ thuật sau:

1. Trang bìa là "Cấu trúc cố định" (Fixed Schema), còn Thân bài là "Cấu trúc động, đệ quy" (Dynamic Tree)

- **Trang bìa (Front Matter):**
  - Luôn gồm các trường cố định: Tiêu đề bài báo, danh sách Tác giả, Cơ quan công tác, Tóm tắt (Abstract), Từ khóa (Keywords), DOI, Ngày gửi/nhận.
  - Vì số lượng và vai trò các ô chứa là **cố định**, ta hoàn toàn tạo được các macro sẵn trong `.cls` kiểu:

    ```
    \spTitle{Đánh giá nồng độ...}
    \spAuthor{Nguyễn Văn A}{Khoa CNTT}
    \spAbstract{Bài viết trình bày...}
    ```

    Đây chính xác là cơ chế "đục lỗ tiêm nội dung".
- **Thân bài (Body):**
  - Không bao giờ cố định. Bạn không thể biết trước bài báo sẽ có bao nhiêu đoạn văn, bao nhiêu hình, bảng nằm ở đâu.
  - Thân bài có cấu trúc **phân cấp và đệ quy**: Trong Section 1 có Tiểu mục 1.1; trong Tiểu mục 1.1 có một Đoạn văn; sau đó là một Bảng; trong một ô của Bảng lại có Danh sách gạch đầu dòng (`itemize`); trong gạch đầu dòng lại chứa Công thức toán nội dòng `$x^2$`.
  - Vì độ sâu và thứ tự xuất hiện là vô tận, **không một template tĩnh nào đục lỗ sẵn được** để bạn tiêm vào. Cách duy nhất là phải dùng một bộ sinh (Emitter/Compiler) để duyệt cây và sinh ra mã lệnh tương ứng.

2. Sự phức tạp của Bảng và Hình ảnh (Floating Environment)

Trong LaTeX, bảng và hình ảnh không phải chỉ là "dán vào":

- **Với Bảng:**
  - Bảng nhỏ nằm gọn 1 cột (`\begin{table}`).
  - Bảng quá dài phải ngắt trang và lặp lại dòng tiêu đề cột (`\begin{longtable}`).
  - Bảng nhiều cột nằm ngang (`\begin{sidewaystable}`).
  - Các ô còn bị gộp dòng/gộp cột (`\multirow`, `\multicolumn`).
  - Một template `.cls` tĩnh không thể dự đoán bảng của tác giả có 3 cột hay 12 cột, có tràn trang hay không để mà viết macro cố định. Bắt buộc code TypeScript phải tính toán lưới (grid) rồi sinh ra đoạn mã TeX tương ứng.
- **Với Hình ảnh:**
  - Hình đơn, hình đôi (2 panel A-B ghép ngang), hay hình tràn cả 2 cột trang báo (`\begin{figure*}`). Tùy vào kích thước và thuộc tính của khối hình mà cú pháp sinh ra TeX sẽ khác nhau.

3. Vấn đề "Lọc rác" và "Bảo mật" (Sanitization & Escaping)

Nếu bạn chỉ "tiêm nội dung thô" vào một template có sẵn:

- Tác giả gõ trong nội dung: `"Tỉ lệ tăng 50% & đạt giá trị cực đại tại x_1"`.
- Nếu tiêm thẳng chuỗi đó vào file TeX:
  - Ký tự `%` sẽ biến toàn bộ phần còn lại của dòng thành chú thích (comment) → mất chữ.
  - Ký tự `&` trong LaTeX là lệnh chuyển cột → gây sập trình biên dịch ngay lập tức.
  - Dấu `_` (underscore) không nằm trong môi trường toán sẽ báo lỗi nghiêm trọng.
- Do đó, hệ thống không thể tiêm chuỗi thô. Toàn bộ chữ trong đoạn văn phải được bóc tách thành các hạt (Inline node), lọc qua hàm an toàn (`escapeTex`), hạt nào là công thức thì bọc trong `$ $`, hạt nào là trích dẫn thì bọc trong `\cite{}`.

4. Mối quan hệ thực tế giữa `.cls` và `LatexExporter`

Thực ra đề xuất kiến trúc đã phân công nhiệm vụ rất chuẩn:

Thành phần	Ai phụ trách?	Bản chất là gì?	Làm nhiệm vụ gì?
File `.cls` / `.sty`	Người làm LaTeX chuyên nghiệp (Typesetter)	Khuôn mẫu & Quy tắc hiển thị	Định nghĩa các macro trang bìa, thiết kế lề trang, màu sắc khung tóm tắt, font chữ tiêu đề mục (`\section`), kiểu số trang header/footer.
`LatexExporter` (TypeScript)	Lập trình viên hệ thống	Động cơ chuyển dịch cấu trúc (AST → TeX)	Đọc cây Manuscript; tiêm thông tin vào các macro trang bìa của file `.cls`; sau đó **tự động biến đổi từng khối thân bài** (đoạn văn, bảng, hình, công thức) thành các khối lệnh LaTeX tương thích với `.cls` đó.

Nói một cách hình ảnh:

- **File `.cls`** giống như việc ta **xây sẵn móng nhà, cột nhà, mái nhà và các quy định về màu sơn**.
- Còn **Thân bài (Body)** là **đồ đạc nội thất** do người dùng mang tới (có nhà 3 cái bàn, có nhà 10 cái ghế). Hệ thống TypeScript chính là người thợ sắp xếp từng món đồ vào đúng phòng, chứ không thể đúc sẵn khuôn cho những món đồ chưa biết trước hình dáng.

User prompt: Liệu là lúc biên dịch latex có cách nào để tối ưu thời gian biên dịch latex ko á, tại tôi thấy nó sẽ tốn một ít thời gian

Response: Biên dịch LaTeX (đặc biệt là các engine hiện đại như XeLaTeX hay LuaLaTeX) thường tốn từ 2 đến 6 giây cho một tài liệu học thuật có font OpenType, nhiều công thức và hình ảnh. Với một Web Editor, độ trễ này sẽ làm đứt mạch trải nghiệm nếu biên dịch liên tục sau mỗi thao tác gõ.

Để tối ưu triệt để thời gian biên dịch từ vài giây xuống mức phần trăm giây (sub-second) hoặc tối ưu cảm giác chờ đợi của người dùng, bạn có thể triển khai kết hợp 5 giải pháp kiến trúc sau:

1. Phân tầng Preview (Kiến trúc 2 tầng) — Giải pháp cốt lõi

Đừng bắt trình biên dịch TeX phải gánh việc "hiển thị khi đang gõ phím". Hãy tách rõ 2 tầng:

- **Tầng 1 - Instant Preview (Tức thì / < 50ms):** Sử dụng chính cây AST của bài báo để render thẳng ra HTML/CSS bằng React. Công thức toán dùng **KaTeX** render trực tiếp trên trình duyệt. Người dùng gõ đến đâu chữ và công thức hiện ngay đến đó mà không cần gọi xuống backend.
- **Tầng 2 - Accurate PDF Preview (Chính xác / 1–3s):** Chỉ gọi compile LaTeX thật sự khi:
  - Người dùng bấm nút **Biên dịch** thủ công.
  - Hoặc người dùng ngừng gõ phím khoảng 1.5 - 2 giây (Debounce) và có chuyển tab sang chế độ xem PDF.

2. Dùng Precompiled Preamble (Định dạng `.fmt`) — Giảm 60–80% thời gian chạy

Trong một file LaTeX, phần ngốn thời gian nhất không phải là vài trang văn bản của tác giả, mà là việc **nạp hàng chục package nặng** ở phần mở đầu (Preamble): `fontspec`, `polyglossia`, `amsmath`, `hyperref`, `tikz`, `geometry`... Mỗi lần chạy, TeX engine phải đọc và phân giải lại toàn bộ macro này từ đĩa.

- **Cách làm:** Đóng gói toàn bộ các package cố định và cấu hình lớp `.cls` thành một file binary định dạng trước (precompiled format file `.fmt`).

```
# Biên dịch sẵn preamble thành file .fmt một lần duy nhất
xelatex -ini -jobname="structpaper-hs" "&xelatex preamble.tex\dump"
```

- **Khi biên dịch bài viết:** Chỉ cần nạp file `.fmt` đã nạp sẵn vào bộ nhớ:

```
xelatex "&structpaper-hs" article.tex
```

- **Hiệu quả:** Cắt giảm ngay lập tức 1 – 2 giây thời gian phân giải package, lệnh chạy bắt đầu ngay từ phần nội dung thân bài.

3. Server-side Caching theo Hash nội dung (Content-Addressed Cache)

Trong quá trình soạn thảo, phần lớn các lần người dùng bấm biên dịch chỉ thay đổi một vài từ hoặc một đoạn văn ngắn, trong khi các tài nguyên khác (ảnh, font, bib) hoàn toàn không đổi.

- **Asset Caching:** Không gửi lại toàn bộ file ảnh qua mạng. Chỉ gửi danh sách `sha256` của ảnh. Server lưu các ảnh này trong thư mục cache dùng chung.
- **Result Caching:** Tạo hash khóa từ: `hash(nội dung tex + danh sách hash assets + phiên bản profile/cls)`. Nếu khóa hash đã tồn tại trong Redis hoặc thư mục tạm, trả về ngay lập tức file PDF có sẵn mà không cần chạy lại TeX Live.

4. Dùng Ramdisk (Tmpfs) cho thư mục làm việc của Worker

LaTeX là công cụ tạo ra cực kỳ nhiều file phụ trợ trong lúc chạy (`.aux`, `.log`, `.out`, `.toc`, `.fls`, `.synctex.gz`). Việc đọc/ghi các file này liên tục xuống ổ cứng (ngay cả SSD) đều tạo overhead I/O lớn khi có nhiều request đồng thời.

- Cấu hình thư mục build của worker nằm trên **tmpfs** (bộ nhớ RAM ảo của Linux):

```
mount -t tmpfs -o size=256M tmpfs /tmp/latex-builds
```

- Toàn bộ thao tác ghi file trung gian diễn ra trực tiếp trên RAM, triệt tiêu hoàn toàn thời gian chờ I/O đĩa.

5. Cân nhắc Engine biên dịch: Tectonic hoặc LuaLaTeX Cache

Nếu kiến trúc cho phép điều chỉnh engine:

- **Tectonic Engine:** Đây là một bản phân phối TeX viết lại bằng Rust dựa trên XeTeX. Tectonic tự động tối ưu hóa việc nạp font, chỉ đọc các phần cần thiết của package và gom việc chạy nhiều lượt (re-run do `\ref`, `\cite`) vào trong cùng một tiến trình tối ưu, tránh việc phải chạy `xelatex` lặp lại 2-3 lần bằng shell script bên ngoài.
- **Lưu ý Font Cache của XeTeX/fontspec:** Nếu vẫn dùng `xelatex`, đảm bảo container hoặc worker đã chạy lệnh `fc-cache -fv` từ lúc build Docker image. Nếu không có font cache sẵn, mỗi lần nạp `fontspec`, hệ thống sẽ quét lại toàn bộ thư viện font của OS, gây chậm thêm 1–3 giây.

User prompt: cũng ok á, tuy nhiên là nên tìm cách tối ưu thời gian ha, chứ còn việc hiển thị tạm cho người dùng thì ko nên đâu, tại html nó ko có chính xác 100% kết quả như preview pdf thì hiện lên để làm gì đâu, người dùng chỉ hoang mang thêm thôi

Response: Quan điểm này hoàn toàn chính xác với đặc thù của phần mềm dàn trang học thuật (Typesetting / Prepress). Trong các bài báo khoa học, sự chính xác về vị trí ngắt dòng, ngắt trang (widow/orphan), ngắt cột, kích thước ảnh và dàn trang bảng biểu là tiêu chí quan trọng nhất. Nếu bản HTML hiển thị lệch một chút so với bản in PDF thật, người dùng sẽ mất niềm tin vào công cụ.

Khi đã xác định **chỉ hiển thị một kết quả duy nhất là PDF thật từ LaTeX engine**, để tối ưu thời gian phản hồi (chỉ còn khoảng **500ms – 1s** thay vì 4 – 6s), dây chuyền biên dịch backend cần áp dụng các kỹ thuật sâu dưới đây:

1. Precompiled Format File (`.fmt`) — Cắt giảm 60% thời gian khởi động

Lý do lớn nhất khiến XeLaTeX/LuaLaTeX chậm là phải phân giải lại hàng chục package nặng (`fontspec`, `polyglossia`, `amsmath`, `hyperref`, `microtype`, `geometry`...) mỗi khi có lệnh chạy.

- **Cách xử lý:** Đóng gói toàn bộ các package tĩnh và file định dạng của tạp chí (`.cls`) thành một file binary định dạng trước (precompiled `.fmt`).

```
# Khởi tạo trước một lần duy nhất lúc build Docker image:
xelatex -ini -jobname="structpaper-core" "&xelatex preamble.tex\dump"
```

- **Khi có lệnh từ người dùng:** XeTeX chỉ cần nạp thẳng file nhị phân đó vào RAM:

```
xelatex -fmt=structpaper-core article.tex
```

- **Kết quả:** Bỏ qua toàn bộ giai đoạn nạp package, trình biên dịch bắt tay ngay vào việc dàn chữ và xuất trang.

2. Triệt tiêu số lượt chạy (Pass Reduction) nhờ số thứ tự cấp sẵn

Thông thường khi biên dịch một bài báo, người ta phải chạy XeLaTeX **2 đến 3 lần liên tiếp** (`xelatex` → `bibtex` → `xelatex` → `xelatex`) để giải quyết các nhãn `\label`, `\ref`, `\cite` ghi trong file `.aux`. Mỗi lần chạy lại nhân đôi thời gian.

- **Trong StructPaper v2:** Do đã có hàm `computeNumbering` ở tầng TypeScript, toàn bộ số thứ tự của Hình, Bảng, Công thức và Trích dẫn `[n]` đã được **tính sẵn và in thẳng vào mã nguồn TeX**.
- Không cần LaTeX phải chạy lượt 1 để lưu nhãn rồi chạy lượt 2 để điền nhãn.
- **Kết quả:** Hệ thống chỉ cần chạy **đúng 1 pass duy nhất** là ra luôn file PDF hoàn chỉnh, cắt giảm ngay 50–70% tổng thời gian.

3. Tận dụng bộ nhớ RAM (Tmpfs) và Font Cache

Quá trình biên dịch tạo ra rất nhiều file rác trung gian (`.aux`, `.log`, `.out`, `.toc`, `.fls`).

- **Mount Tmpfs:** Cấu hình thư mục làm việc của worker chạy trên RAM ảo:

  ```
  mount -t tmpfs -o size=512M tmpfs /tmp/latex-builds
  ```

  Tốc độ đọc/ghi trên RAM nhanh gấp hàng chục lần so với SSD, loại bỏ tình trạng thắt cổ chai I/O khi nhiều người cùng bấm biên dịch.

- **Nạp sẵn Font Cache:** Khi dùng font tùy biến qua `fontspec`, XeTeX bắt buộc phải quét hệ thống font. Cần chạy lệnh nạp cache tĩnh ngay khi tạo image:

  ```
  RUN fc-cache -f -v
  ```

  Nếu thiếu bước này, XeTeX sẽ quét lại toàn bộ thư mục font của Linux trong mỗi lần chạy, gây trễ thêm 1–2 giây vô ích.

4. Đẩy trước tài nguyên tĩnh (Asset Pre-warming)

Phần lớn dung lượng gửi sang worker là file ảnh (TIFF, PNG, JPG độ phân giải cao).

- Mỗi bức ảnh khi người dùng kéo thả vào bài viết sẽ được gửi ngầm lên server lưu trữ theo mã `sha256`.
- Khi bấm nút biên dịch, client **chỉ gửi file `.tex` kèm danh sách mã hash**. Server chỉ việc gắn liên kết mềm (symlink) từ kho ảnh sẵn có vào thư mục tmpfs để TeX đọc.
- Không tốn thời gian upload lại ảnh mỗi lần bấm biên dịch.

5. Cơ chế Hủy tác vụ cũ (Job Cancellation)

Khi người dùng liên tục chỉnh sửa và bấm biên dịch, các job cũ chưa chạy xong sẽ làm nghẽn hàng đợi (queue) của worker.

- Mỗi bản thảo chỉ giữ tối đa **1 job đang chạy**.
- Khi có một lệnh biên dịch mới được phát đi từ cùng một bài viết, backend lập tức gửi tín hiệu `SIGKILL` để dừng ngay tiến trình XeTeX cũ, giải phóng luồng CPU cho job mới nhất.

User prompt: Ingest DOCX là pipeline nhiều tầng: ưu tiên đọc theo style Word của template, sau đó heuristic, cuối cùng mới AI. Chạy trong Web Worker và viết dạng isomorphic để dùng lại cho batch phía server. ý này là sao vậy

Response: Câu này mô tả **chiến lược đọc và bóc tách file Word (`.docx`)** khi người dùng tải lên, giải quyết dứt điểm tình trạng parser cũ bị phình to hơn 2.100 dòng code và hay đoán mò làm sai định dạng.

Để dễ hiểu, ý này được chia thành 3 phần chính sau:

1. "Pipeline nhiều tầng" (Đường ống xử lý từng bước)

Thay vì nhồi toàn bộ logic (mở file zip, đọc XML, gỡ bảng, bóc ảnh, dịch công thức toán, nhận diện tác giả) vào chung một hàm khổng lồ, quá trình đọc file được chia thành một **dây chuyền sản xuất** gồm 6 trạm độc lập:

1. **Mở gói (Package):** Giải nén file `.docx` để lấy các file XML và thư mục ảnh bên trong.
2. **Đọc thô (OOXML → RawDoc):** Duyệt XML bằng một bộ phân tích thật để lấy ra từng đoạn văn bản, bảng biểu, công thức mà chưa cần gán ý nghĩa.
3. **Nhận diện vai trò (Recognizers):** Xác định xem khối này là Tiêu đề, Tác giả, Tóm tắt hay Hình ảnh.
4. **Lắp ráp (Assemblers):** Gom các khối đã nhận diện thành cấu trúc bài báo hoàn chỉnh.
5. **Chuẩn hóa (Normalizers):** Sửa lỗi gõ tiếng Việt (NFC/NFD), sửa khoảng trắng, nối các trích dẫn `[1, 2]`.
6. **Kiểm tra (Validate):** Đảm bảo dữ liệu đúng chuẩn và xuất ra báo cáo các chỗ còn nghi ngờ.

2. "Ưu tiên đọc theo style Word → sau đó heuristic → cuối cùng mới AI"

Đây là **thứ tự ưu tiên từ rẻ nhất, chính xác nhất đến đắt và rủi ro nhất** để nhận diện từng phần trong bài viết:

- **Tầng 1 - Style Word của template (Độ tin cậy 100%):** Các file mẫu của tòa soạn (`Template_EN.docx`, `Template_VI.docx`) đều đã định nghĩa sẵn các kiểu dáng (Styles) chuẩn như `ArticleTitle`, `Authors`, `Abstract`, `Heading1`, `TableCaption`... Nếu tác giả tải file mẫu về và gõ đúng vào các khung đó, hệ thống chỉ cần đọc tên Style trong file Word là biết ngay đoạn đó là gì mà **không cần suy đoán bất cứ điều gì**.
- **Tầng 2 - Heuristic (Luật phỏng đoán theo kinh nghiệm - Độ tin cậy 80%):** Nếu tác giả không dùng template chuẩn mà tự định dạng bằng tay, hệ thống sẽ dùng các tập luật logic để suy luận:
  - Thấy dòng bắt đầu bằng `"Bảng 1:"` hoặc `"Table 2."` → đoán đây là chú thích bảng (`TableCaption`).
  - Thấy dòng bắt đầu bằng `"Từ khóa:"` hoặc `"Keywords:"` → đoán đây là mục từ khóa.
  - Thấy đoạn văn nằm ngay đầu bài, cỡ chữ rất to, in đậm, căn giữa → nhiều khả năng là Tiêu đề bài báo.
- **Tầng 3 - AI (Giải pháp cứu cánh cuối cùng - Độ tin cậy ~70%, cần người duyệt lại):** Chỉ những phần quá lộn xộn hoặc thông tin tác giả/cơ quan công tác bị viết lẫn lộn mà 2 bước trên bó tay thì mới gửi đoạn văn đó lên mô hình AI để nhờ bóc tách. Cách này vừa **tiết kiệm tiền gọi API**, vừa **tránh rủi ro AI tự bịa nội dung**.

3. "Chạy trong Web Worker và viết dạng isomorphic để dùng lại cho batch phía server"

- **Chạy trong Web Worker (Không làm đơ trình duyệt):**
  - Bình thường, các tác vụ tính toán nặng trên trang web sẽ chạy chung một luồng (thread) với giao diện người dùng. Khi nạp một file Word dài cả trăm trang chứa nhiều bảng biểu và ảnh dung lượng lớn, trình duyệt sẽ bị khựng, chuột quay tròn, nút bấm không phản hồi.
  - Đưa bộ bóc tách vào **Web Worker** tức là đẩy toàn bộ việc phân tích file Word sang một luồng chạy ngầm phía dưới. Trình duyệt vẫn mượt mà, người dùng vẫn bấm qua lại các tab được bình thường.
- **Isomorphic (Chạy được ở cả hai đầu: Trình duyệt và Máy chủ):**
  - Code của bộ đọc file Word được viết hoàn toàn bằng TypeScript chuẩn, **không sử dụng bất kỳ hàm đặc thù nào của trình duyệt** (như không dùng `document`, `window`, hay `DOMParser`).
  - Nhờ vậy, **cùng một đoạn code đó**:
    - Khi người dùng upload 1 file đơn lẻ trên web → code chạy ngay trên trình duyệt của họ.
    - Khi tòa soạn cần xử lý hàng loạt (**batch processing**) cả trăm bài báo gửi về cùng lúc dạng file `.zip` → server (Node.js) lấy đúng thư viện đó ra để bóc tách tự động mà **không phải viết lại một bộ đọc file Word khác bằng Java hay Python**.

User prompt: Nếu mà làm theo tài liêu đề xuất đó thì liệu là nhiều người trong team có thể dễ dàng cộng tác với nhau không. Ví dụ như là có nhiều hạng mục cần xử lý, bảng, hình, equation,... thì có thể chia nhau từng phần như vậy không

Response: **Hoàn toàn có thể, và đây chính là mục tiêu thiết kế cốt lõi nhất của tài liệu đề xuất này.**

Trước đây với bản v1, cả team **không thể làm song song** vì tất cả mọi thứ bị nhồi vào hai file khổng lồ: `docx-parser.ts` (>2.100 dòng) và `UnifiedEditorView.tsx`. Chỉ cần một người vào sửa logic bảng là rất dễ gây xung đột Git (conflict) hoặc làm gãy code phần hình ảnh của người khác.

Bản đề xuất v2 giải quyết bài toán chia việc bằng cách **cắt hệ thống thành các khối độc lập theo cả chiều ngang lẫn chiều dọc**:

1. Phân chia rạch ròi theo từng hạng mục (Cắt dọc theo Feature)

Nhờ cơ chế **Block Registry** (mục 6.3) và cấu trúc module, mỗi thành phần như **Bảng (Table)**, **Hình (Figure)**, **Công thức (Equation)**, **Tài liệu tham khảo (Reference)** sẽ trở thành một "gói công việc độc lập" từ đầu đến cuối.

Ví dụ, nếu team phân công:

- **Bạn A nhận phần Bảng (Table):** Bạn A chỉ làm việc trong các file riêng biệt sau mà không cần đụng vào file của người khác:
  - Đọc bảng từ Word: `packages/ingest-docx/assemblers/table.ts`
  - Giao diện sửa bảng: `packages/feature-body/table/`
  - Xuất bảng sang LaTeX: `packages/exporters/latex/blocks/table.ts`

- Luật kiểm tra lỗi bảng (QC): `packages/qc-engine/rules/table.ts`

- **Bạn B nhận phần Công thức (Equation):** Bạn B toàn quyền xử lý:
  - Chuyển OMML → MathML → LaTeX: `packages/ingest-docx/assemblers/equation.ts`
  - Giao diện nhập công thức (MathLive/KaTeX): `packages/feature-body/equation/`
  - Xuất công thức sang LaTeX: `packages/exporters/latex/blocks/equation.ts`
- **Bạn C nhận phần Hình ảnh (Figure):**
  - Nhận diện ảnh, tính DPI: `packages/ingest-docx/assemblers/figure.ts`
  - Giao diện quản lý ảnh: `packages/feature-body/figure/`
  - Xuất ảnh sang LaTeX: `packages/exporters/latex/blocks/figure.ts`

Lợi ích: Ba người mở 3 thư mục hoàn toàn khác nhau, commit lên các nhánh Git khác nhau, khi merge code **xác suất conflict gần như bằng 0**.

2. Phân chia theo tầng chuyên môn (Cắt ngang theo Layer)

Nếu trong nhóm có bạn giỏi giao diện, bạn giỏi thuật toán, bạn rành LaTeX, bạn lại chuyên Backend, kiến trúc này cho phép chia việc theo thế mạnh:

Nhóm việc	Chuyên môn phù hợp	Phạm vi code	Trách nhiệm
Doc Model & Ingest	Giỏi thuật toán, phân tích XML	`packages/doc-model`, `packages/ingest-docx`	Định nghĩa chuẩn dữ liệu, dựng bộ bóc tách file Word. Chỉ cần giao ra đúng cấu trúc `Manuscript v2`.
Editor UI	Giỏi React, Frontend	`packages/editor-core`, `packages/feature-*`	Dựng màn hình soạn thảo, thanh công cụ, các modal nhập liệu. Không cần quan tâm file Word được parse thế nào hay LaTeX chạy ra sao.
LaTeX & Prepress	Rành TeX, thiết kế dàn trang	File `.cls`, `packages/exporters/latex`	Tinh chỉnh mẫu hiển thị của tạp chí, viết hàm biến đổi từ AST sang câu lệnh LaTeX chuẩn đẹp.
Backend & Worker	Giỏi Spring Boot, Docker, Linux	`apps/api`, `services/compiler-worker`	Dựng API lưu trữ, thiết lập container sandbox biên dịch LaTeX có hàng đợi (queue) và cache RAM (tmpfs).

3. Điều kiện tiên quyết để phối hợp mượt mà: "Chốt hợp đồng trước"

Để nhiều người làm song song mà khi ghép lại chạy được ngay, nhóm chỉ cần tuân thủ đúng lộ trình:

1. **Làm xong Giai đoạn 1 trước:** Chốt cứng cấu trúc dữ liệu `Manuscript v2` và Zod Schema trong package `doc-model`.
2. **Ký hợp đồng:** Khi cấu trúc dữ liệu của một `TableBlock` hay `EquationBlock` đã được định nghĩa rõ ràng thì:
   - Người làm Ingest biết mình phải trả ra dữ liệu dạng gì.
   - Người làm Editor biết mình cần hiển thị những trường nào.
   - Người làm Exporter biết mình cần đọc thuộc tính nào để sinh ra mã LaTeX.

Mọi người có thể code độc lập, viết Unit Test dựa trên dữ liệu mẫu (mock data), đến ngày tích hợp chỉ việc ráp các mảnh ghép lại là hoàn chỉnh.

User prompt: Block registry là gì vậy? ủa mà những file table.ts, ... rời rạc như vậy thì mai mốt làm sao để chúng kết hợp lại vậy, trong tài liệu có đề cập đến không

Response: 1. Block Registry là gì?

Registry (sổ đăng ký) là một pattern (mẫu thiết kế) đóng vai trò như một **"danh bạ trung tâm"**.

Nói một cách dân dã: Bạn có một ổ cắm điện đa năng và nhiều thiết bị (quạt, đèn, nồi cơm). Muốn dùng thiết bị nào, bạn chỉ cần "cắm" thiết bị đó vào ổ cắm. **Block Registry** chính là cái ổ cắm đó:

- Nó là một bảng kê khai lưu trữ: *"Hệ thống hiện tại hỗ trợ những loại khối (block) nào? Khi gặp khối đó thì dùng giao diện nào để sửa, dùng hàm nào để xuất ra LaTeX, dùng hàm nào để xuất ra Word?"*

2. Các file rời rạc (`table.ts`, `equation.ts`, `figure.ts`...) kết hợp lại thế nào?

Tài liệu **có đề cập trực tiếp đến cơ chế này ở Mục 6.3 (Feature module và Block registry)**.

Cơ chế kết hợp hoạt động theo 3 bước mạch lạc sau:

Bước 1: Khai báo "bộ hồ sơ" cho từng khối (Đăng ký vào Registry)

Mỗi thành phần (ví dụ `table`) sẽ gom toàn bộ các hàm của mình lại và nộp vào Registry thông qua hàm `registerBlock`:

```
// Nằm ở file định nghĩa khối table
registerBlock('table', {
  schema: TableBlockSchema,       // Cấu trúc dữ liệu của bảng
  Editor: TableEditor,             // Giao diện React để bấm sửa ô, gộp cột
  toLatex: tableToLatex,           // Hàm biến bảng thành lệnh \begin{table}...
  toDocx: tableToDocx,             // Hàm biến bảng thành thẻ XML Word
  toJats: tableToJats,             // Hàm biến bảng thành thẻ XML JATS
  qc: tableRules                   // Các luật kiểm tra lỗi bảng (ví dụ: thiếu chú thích)
});
```

Tương tự, người làm `figure` hay `equation` cũng đăng ký đúng mẫu như vậy với nhãn `'figure'` và `'equation'`.

Bước 2: Khi chạy, hệ thống chỉ việc "tra danh bạ" theo loại khối

Khi hệ thống cần làm một việc gì đó (ví dụ: hiển thị bài báo lên màn hình, hoặc xuất ra file LaTeX), nó không cần viết hàng chục câu lệnh `if...else` rườm rà. Nó chỉ cần duyệt qua cây bài viết và tra từ registry:

- **Khi Editor cần hiển thị bài báo lên màn hình:** Duyệt qua danh sách block:

  ```
  // Khi gặp block có type = 'table'
  const blockDef = blockRegistry.get(block.type);
  const Component = blockDef.Editor; // Lấy ra TableEditor
  return <Component data={block} />;
  ```

- **Khi xuất ra file LaTeX (`LatexExporter`):** Duyệt qua từng block:

  ```
  const blockDef = blockRegistry.get(block.type);
  const latexCode = blockDef.toLatex(block); // Gọi tableToLatex(block)
  ```

3. TypeScript kiểm soát để không bao giờ bị "sót" (Record<BlockType, Handler>)

Tài liệu có nêu một điểm kỹ thuật then chốt: **TypeScript buộc kiểu `Record<BlockType, Handler>` ở mỗi tầng.**

Nghĩa là:

- Giả sử hệ thống quy định bài báo gồm các loại block: `'paragraph' | 'figure' | 'table' | 'equation'`.
- Nếu một người mới vào team tạo thêm một loại khối mới là `'video'` hoặc `'callout'`, nhưng chỉ viết giao diện mà **quên viết hàm `toLatex`**.
- Trình biên dịch TypeScript sẽ **báo lỗi đỏ ngay lập tức và chặn không cho build** với thông báo: *"Khối 'callout' chưa có hàm toLatex trong danh bạ!"*.

Nhờ cơ chế này:

1. **Ai làm việc nấy:** Người làm bảng chỉ tập trung vào file của bảng, không cần mở file của hình hay công thức.
2. **Không sợ sót:** Trình biên dịch tự động ép buộc các file rời rạc khi ráp vào "ổ cắm" trung tâm phải cung cấp đầy đủ các đầu hàm theo đúng hợp đồng.

User prompt: Journal Profile là dữ liệu (JSON có schema), không phải code. Thêm tạp chí mới = thêm profile và một file .cls. còn ý này là sao vậy

Response: Ý này đề cập đến nguyên lý **"Tách biệt giữa quy tắc nghiệp vụ (Dữ liệu) và cơ chế vận hành (Mã nguồn)"**, nhằm giải quyết triệt để vấn đề sửa code rải rác mỗi khi bổ sung một tạp chí mới.

1. "Journal Profile là dữ liệu (JSON có schema), không phải code" nghĩa là gì?

- **Bản v1 bị gì?** Quy định của từng tạp chí bị code cứng (hard-code) bằng các lệnh rẽ nhánh `if / else` nằm rải khắp giao diện và backend. Ví dụ:
  - Trong tab Tóm tắt: `if (journal === 'HS') { maxWords = 500; } else { maxWords = 250; }`
  - Trong tab Từ khóa: `if (journal === 'HS') { sep = ';'; } else { sep = ','; }`
  - Trong logic xuất LaTeX: `if (journal === 'HS') { font = 'Times'; }`

  Cách làm này khiến mã nguồn ngày càng rối. Cứ mỗi lần một tạp chí đổi quy chế hay thêm tạp chí mới, lập trình viên phải lục tung hàng chục file code để bổ sung nhánh `else if`.

- **Bản v2 làm gì?** Mọi quy định của một tạp chí được gom toàn bộ vào **một file dữ liệu JSON duy nhất**, tuân theo một khuôn mẫu định sẵn (**Schema** được kiểm soát bằng thư viện `Zod`):

  ```
  {
    "id": "VNUHCM-HS",
    "names": { "vi": "Tạp chí Phát triển KH&CN - Khoa học Sức khỏe" },
    "frontMatter": {
      "abstract": { "minWords": 275, "maxWords": 500 },
      "keywords": { "min": 3, "max": 5, "separator": ";" },
      "dateFormat": "dd/MM/yyyy"
    },
    "citation": { "inText": "superscript" },
    "typography": { "fonts": "Times New Roman", "size": 10 },
    "latex": { "cls": "structpaper-hs.cls" }
  }
  ```

  Khi hệ thống chạy, nó chỉ việc đọc file JSON này lên: ô tóm tắt tự biết giới hạn từ, bộ kiểm tra lỗi (QC) tự biết dùng dấu chấm phẩy hay dấu phẩy, exporter tự biết dùng font gì mà **không cần một dòng lệnh `if (journal == ...)` nào trong code**.

2. "Thêm tạp chí mới = thêm profile và một file .cls" nghĩa là gì?

Khi hệ thống cần hỗ trợ một tạp chí mới (ví dụ: *Tạp chí Khoa học Tự nhiên*):

Quy trình trước đây đòi hỏi lập trình viên phải sửa code logic, test lại toàn bộ ứng dụng và deploy lại hệ thống. Với kiến trúc mới, công việc chỉ gói gọn trong **2 file tĩnh**:

1. **Một file cấu hình (`journal-natural-sciences.json`):** Điền các thông số quy định của tạp chí (số từ tóm tắt, kiểu trích dẫn, lề trang, tên tạp chí).
2. **Một file giao diện mẫu LaTeX (`structpaper-ns.cls`):** Chuyên viên dàn trang thiết kế sẵn khung trang bìa, header, footer của tạp chí đó bằng TeX.

Lập trình viên không cần viết lại logic bóc tách, không cần sửa trình biên dịch, không cần sửa Editor. Hệ thống nạp 2 file này vào là tạp chí mới lập tức sẵn sàng hoạt động (tuân thủ nguyên lý **Open/Closed**: Mở rộng tính năng mới mà không sửa đổi mã nguồn cũ).

User prompt: Editor = Store + Command + Feature module + Block registry (undo/redo, autosave, op-log đồng bộ BE có sẵn). ý này là sao vậy

Response: Ý này mô tả **kiến trúc vận hành bên trong của màn hình soạn thảo (Editor)**, giải quyết việc bản cũ bị lỗi mất dữ liệu khi đóng tab, không có Ctrl+Z (Undo/Redo), và code bị dồn cục vào một file khổng lồ (`UnifiedEditorView.tsx`).

Công thức này gồm 4 khối ghép lại với nhau và tự động sinh ra 3 tính năng nâng cao mà không tốn thêm công sức:

1. Bốn thành phần chính cấu thành Editor

- **Store (Kho dữ liệu tập trung):**

- *Bản cũ:* Dữ liệu bị phân mảnh ở nhiều nơi (`formData`, `PaperContext`, `sessionStorage`), cập nhật chỗ này lại quên chỗ kia.
- *Bản mới:* Toàn bộ bài báo (`Manuscript v2`) chỉ nằm ở **một nơi duy nhất** (dùng Zustand + Immer). Mọi màn hình, mọi ô nhập liệu đều nhìn vào kho này.

- **Command (Mệnh lệnh thay đổi):**
  - Giao diện **không bao giờ được phép tự ý gán hay sửa đè dữ liệu** kiểu `paper.title = '...'`.
  - Mọi hành động của người dùng đều phải đóng gói thành một "mệnh lệnh" (Command) gửi đến một bộ điều phối trung tâm (`CommandBus`), ví dụ:
    - Người dùng xóa ảnh → gửi lệnh: `{ type: 'figure/delete', id: 'fig-1' }`.
    - Người dùng chèn đoạn văn → gửi lệnh: `{ type: 'block/insert', content: '...' }`.
- **Feature module (Cắt giao diện theo tính năng):**
  - Không nhét tất cả code vào một màn hình to.
  - Mỗi thẻ/tab trên màn hình soạn thảo là một thư mục riêng biệt: `feature-metadata` (quản lý tiêu đề/tác giả), `feature-references` (quản lý tài liệu tham khảo), `feature-qc` (bảng báo lỗi). Các nhóm này độc lập, ai phụ trách tab nào chỉ mở đúng thư mục đó.
- **Block registry (Danh bạ các loại khối):**
  - Đã giải thích ở câu trước: Bảng, Hình, Công thức, Đoạn văn là các module độc lập được đăng ký vào danh bạ trung tâm để Editor tự động nạp khi cần.

2. "undo/redo, autosave, op-log đồng bộ BE có sẵn" — Lợi ích tự động sinh ra

Khi đã bắt mọi thay đổi phải đi qua **Store** và **Command**, hệ thống lập tức có được 3 tính năng phức tạp một cách "miễn phí":

- **Undo / Redo (Ctrl+Z / Ctrl+Y) miễn phí:**
  - Nhờ dùng thư viện `Immer`, mỗi khi một Command chạy xong, nó không chỉ tạo ra dữ liệu mới mà còn tự động sinh ra một "bản vá ngược" (Inverse Patch — ví dụ: vừa xóa một khối thì bản vá ngược là lệnh khôi phục lại khối đó).
  - Khi bấm `Ctrl+Z`, hệ thống chỉ việc áp bản vá ngược đó vào Store là bài viết quay lại trạng thái cũ ngay lập tức.
- **Autosave (Tự động lưu ngầm không đơ máy):**
  - Hệ thống không cần người dùng bấm `Ctrl+S`.
  - Có một tiến trình ngầm chỉ việc "nghe" các Command vừa phát ra: cứ sau khoảng 1 giây người dùng ngừng gõ, tiến trình này tự động lấy dữ liệu mới nhất ghi xuống cơ sở dữ liệu trình duyệt (`IndexedDB`). Tắt trình duyệt hoặc cúp điện đột ngột bật lại vẫn còn nguyên.
- **Op-log đồng bộ BE (Nhật ký thao tác sẵn sàng cho Backend):**
  - Thay vì mỗi lần lưu là phải gửi cả bài báo dung lượng lớn (chứa đầy văn bản) lên server qua mạng, hệ thống chỉ cần gom các Command (gọi là operation log - op-log, ví dụ: *"vừa sửa chữ A thành B ở đoạn 3"*) đẩy lên máy chủ.
  - Giúp tiết kiệm băng thông mạng và sau này dễ dàng mở rộng sang tính năng nhiều người cùng chỉnh sửa một bài viết.

User prompt: Immer là gì vậy 2. ủa vậy kiến trúc này có đề cập đến việc nên lưu track change đó vào be hay không á 3. với lại các track change thay đổi đó trên trình duyệt lưu ở đâu mà undo, redo có thể hoạt động đc vậy

Response: 1. Immer là gì?

Immer là một thư viện JavaScript/TypeScript giúp bạn **sửa đổi dữ liệu phức tạp một cách tự nhiên nhưng vẫn giữ được tính bất biến (Immutability)**.

- **Vấn đề thông thường:** Trong React, bạn không được sửa trực tiếp biến trạng thái (`state.title = 'mới'`) mà phải sao chép nông qua spread operator (`{ ...state, meta: { ...state.meta, title: 'mới' } }`). Khi bài báo có cấu trúc cây lồng nhau sâu 4–5 tầng, code copy này cực kỳ dài dòng và rất dễ sót.

- **Cách Immer giải quyết:** Immer cho bạn một bản nháp tạm thời (`draft`). Bạn cứ thao tác tự nhiên như code bình thường:

  ```
  // Cứ gán thẳng như code JS thuần
  draft.sections[0].blocks[2].content = "Nội dung mới";
  ```

  Immer sẽ tự động theo dõi bạn vừa chạm vào đâu để sinh ra một cây dữ liệu hoàn toàn mới ở đầu ra.

- **Vũ khí quan trọng nhất của Immer trong dự án này:** Tính năng **Patches**. Mỗi khi bạn sửa `draft`, Immer tự động xuất ra 2 danh sách lệnh:
  - `patches`: Mô tả bạn vừa làm gì (ví dụ: *thay chuỗi ở vị trí X bằng "Nội dung mới"*).
  - `inversePatches`: Lệnh đảo ngược 180 độ việc bạn vừa làm (ví dụ: *thay chuỗi ở vị trí X về lại "Nội dung cũ"*). Đây chính là chìa khóa để làm Undo/Redo.

2. Kiến trúc có đề cập đến việc lưu Track Changes (lịch sử thay đổi) vào Backend hay không?

Có, tài liệu nêu rất chi tiết và chia thành 2 khái niệm / 2 giai đoạn rõ ràng:

A. Phân biệt 2 loại "Track Changes":

1. **Track Changes từ file Word nộp lên (Mục 4 - Tầng 2b):**
   - Nếu file Word của tác giả có bật tính năng *Track Changes* của Microsoft Word (các đoạn gạch đỏ, chữ xanh thêm bớt), hệ thống có chính sách xử lý ngay lúc đọc (chấp nhận hết `acceptAll`, từ chối hết `rejectAll`, hoặc giữ lại làm bình luận).
2. **Lịch sử chỉnh sửa trên hệ thống (Revisions / Op-log):**
   - Đây chính là chuỗi thay đổi phát sinh khi người dùng gõ trên Editor.

B. Chiến lược lưu vào Backend (Mục 9.4 và Mục 10.2):

Tài liệu đề xuất lộ trình 2 giai đoạn (`S1` → `S2`) để đội ngũ không bị quá tải:

- **Giai đoạn đầu (S1 - Đơn giản):** Backend chỉ lưu bản chụp toàn bộ bài báo (**Snapshot**) kèm số phiên bản (`rev`) vào bảng `manuscripts` và bảng `manuscript_revisions`. Lưu theo cơ chế khóa lạc quan (`If-Match: <rev>`). Lúc này Backend chưa cần phân tích từng thao tác nhỏ.
- **Giai đoạn sau (S2 - Chi tiết):** Toàn bộ chuỗi thao tác (Operation log / Patches sinh ra từ Command) sẽ được đẩy lên lưu trữ chi tiết ở cột `payload` của bảng `manuscript_revisions` trên MySQL. Nhờ đó, Backend biết rõ ai đã sửa từ nào, lúc mấy giờ, và hỗ trợ tính năng xem lại lịch sử phiên bản (Version History) như Google Docs.

3. Các thay đổi đó trên trình duyệt lưu ở đâu để Undo / Redo hoạt động được?

Trên trình duyệt, hệ thống chia việc lưu trữ làm **2 tầng tách biệt**: một tầng nằm trên **RAM** để phục vụ Undo/Redo tức thì, và một tầng nằm dưới **ổ cứng (IndexedDB)** để chống mất dữ liệu khi tắt máy.

Tầng 1: Ngăn xếp trên bộ nhớ RAM (In-Memory History Stack) — Nơi trực tiếp chạy Undo/Redo

Trong bộ điều khiển trung tâm (`editor-core`), hệ thống duy trì 2 danh sách mảng (Stack) nằm ngay trên RAM của trình duyệt:

- **`undoStack`**: Chứa danh sách các `inversePatches` (các bản vá lùi).
- **`redoStack`**: Chứa danh sách các `patches` (các bản vá tiến).

Cơ chế hoạt động:

1. Khi bạn gõ chữ hoặc xóa ảnh: Một lệnh (`Command`) được gửi đi. Immer sinh ra bản vá đảo ngược và đẩy vào `undoStack`.
2. Khi bạn bấm **Ctrl + Z (Undo)**: Hệ thống rút lệnh gần nhất từ `undoStack` ra, áp dụng ngược lại vào bài viết → chữ vừa gõ biến mất ngay tức thì (dưới 5 mili-giây vì đọc trực tiếp trên RAM). Đồng thời, đẩy lệnh đó sang `redoStack`.
3. Khi bạn bấm **Ctrl + Y (Redo)**: Rút từ `redoStack` ra và áp dụng xuôi trở lại.

Tầng 2: Bảng `revisions` trong IndexedDB (Ổ cứng trình duyệt) — Nơi lưu bền vững

RAM sẽ bị xóa sạch nếu người dùng bấm F5 hoặc lỡ tay tắt tab. Để giải quyết việc này:

- Tài liệu thiết kế bảng `revisions` bên trong cơ sở dữ liệu **Dexie / IndexedDB** của trình duyệt (Mục 9.2).
- Cứ sau một khoảng thời gian ngắn hoặc khi người dùng thực hiện một mốc chỉnh sửa, chuỗi Patch này cùng bản chụp bài viết sẽ được ghi ngầm xuống ổ cứng.
- Khi người dùng mở lại trình duyệt sau khi tắt máy, hệ thống đọc bảng `revisions` từ IndexedDB lên để khôi phục lại trạng thái bài báo mà không bị mất mát dữ liệu.

User prompt: 1. Vấn đề thông thường: Trong React, bạn không được sửa trực tiếp biến trạng thái (state.title = 'mới') mà phải sao chép nông qua spread operator ({ ...state, meta: { ...state.meta, title: 'mới' } }) ý này là sao vậy. Có nghĩa là bài báo lưu ở dạng state hả hay sao mà bạn nói ý đó. Trước đó tôi tưởng là lưu zudstand mà 2. các cơ chế ở trên stack đồ bạn vừa trình bày là của immer hay là chúng ta phải tự triển khai thêm vậy

Response: 1. "State" trong React là gì? Dùng Zustand thì có còn là state không?

Có, Zustand thực chất chính là State.

Nhiều người hay nhầm: tưởng rằng chỉ có `useState` của React mới gọi là state, còn Zustand là một thứ gì đó khác. Nhưng về mặt khái niệm:

- **State (Trạng thái):** Là bất kỳ dữ liệu nào quyết định giao diện hiển thị ra sao và có thể thay đổi theo thời gian (nội dung bài báo, danh sách tác giả, con trỏ đang ở đâu...).
- **`useState`:** Là state cục bộ của **một component nhỏ** (ví dụ: trạng thái đóng/mở của một menu).
- **Zustand:** Là công cụ quản lý state **toàn cục (Global State)** cho toàn bộ ứng dụng. Thay vì nhét dữ liệu vào từng component, Zustand gom toàn bộ dữ liệu bài báo vào một cái kho trung tâm gọi là Store.

Tuy nhiên, dù bạn dùng `useState`, Redux, hay **Zustand**, tất cả chúng đều chạy trong hệ sinh thái React và bắt buộc phải tuân theo một nguyên tắc tối thượng: **Tính bất biến (Immutability)**.

Tại sao không được viết `state.title = 'mới'`?

React (và cả Zustand) nhận biết dữ liệu có thay đổi hay không bằng cách **so sánh địa chỉ ô nhớ (Shallow comparison - so sánh tham chiếu)**:

- Nếu bạn viết `state.title = 'mới'`, nội dung bên trong đổi nhưng **địa chỉ ô nhớ của cả bài báo vẫn y nguyên**. React kiểm tra thấy địa chỉ không đổi nên nó nghĩ: *"Ủa đâu có gì mới đâu"*, và nó **không vẽ lại giao diện (không re-render)**. Kết quả: bạn gõ phím nhưng màn hình đứng im.
- Vì vậy, quy tắc chuẩn là mỗi khi sửa, bạn phải tạo ra một đối tượng hoàn toàn mới ở một vùng nhớ mới.

Nỗi khổ của "Spread operator" `{ ...state }` với dữ liệu lồng nhau:

Một bài báo khoa học có cấu trúc phân tầng rất sâu: `Bài báo` → `Chương (Section)` → `Đoạn văn (Block)` → `Mẩu chữ (Inline)`.

Nếu không có Immer, chỉ để đổi một chữ trong đoạn văn thứ 3 của chương 1, code với Zustand thuần túy sẽ trông khủng khiếp thế này:

```
// Sửa thủ công bằng spread operator {...}
set((state) => ({
  ...state,
  body: state.body.map((sec, secIdx) =>
    secIdx === 0
      ? {
          ...sec,
          blocks: sec.blocks.map((block, bIdx) =>
            bIdx === 2
              ? { ...block, content: [{ t: 'text', v: 'Nội dung mới' }] }
              : block
          )
        }
      : sec
  )
}));
```

Nhìn rất rối mắt, khó đọc và cực kỳ dễ gõ nhầm.

Khi kết hợp **Zustand + Immer** (Zustand có sẵn middleware tích hợp Immer), bạn chỉ cần viết ngắn gọn:

```
set((state) => {
  state.body[0].blocks[2].content[0].v = 'Nội dung mới';
});
```

Bạn gán trực tiếp tự nhiên như JavaScript bình thường, còn Immer sẽ chạy ngầm bên dưới để tự động tạo ra một vùng nhớ mới tinh cho React nhận diện.

2. Các cơ chế Stack (Undo / Redo) là của Immer hay ta phải tự viết?

Câu trả lời: Immer cung cấp "nguyên liệu cốt lõi" (hàm sinh và áp dụng Patch), còn cấu trúc ngăn xếp (Stack) và quy trình điều khiển thì nhóm phải tự viết thêm.

Cụ thể phân chia như sau:

A. Phần Immer làm sẵn cho bạn (Nguyên liệu):

Immer có một module chuyên dụng tên là `produceWithPatches` và hàm `applyPatches`:

1. **Tự động sinh Patch:** Khi bạn chạy một Command (ví dụ: sửa chữ A thành B), Immer tự động xuất ra hai mảng:
    - `patches`: `[{ op: 'replace', path: ['body', 0, 'title'], value: 'B' }]` (Thao tác xuôi).
    - `inversePatches`: `[{ op: 'replace', path: ['body', 0, 'title'], value: 'A' }]` (Thao tác ngược).
2. **Hàm khôi phục:** Immer cung cấp sẵn lệnh `applyPatches(state, inversePatches)` để tự động đắp lại trạng thái cũ mà bạn không cần tự code thuật toán tìm vị trí để gán lại.

B. Phần nhóm phải tự triển khai (Bộ điều khiển):

Immer không tự quản lý lịch sử giúp bạn (nó không tự giữ danh sách phím bấm hay biết khi nào bạn bấm Ctrl+Z). Nhóm sẽ tự viết một class nhỏ trong `packages/editor-core/history.ts`, cấu trúc thực tế chỉ khoảng 40–60 dòng code:

```
// packages/editor-core/history.ts
import { Patch, applyPatches } from 'immer';

export class HistoryManager {
  private undoStack: Patch[][] = [];
  private redoStack: Patch[][] = [];
  private maxHistory = 100; // Giới hạn 100 lần undo để tránh tràn RAM

  // 1. Khi có thao tác mới sinh ra từ Command
  record(patches: Patch[], inversePatches: Patch[]) {
    this.undoStack.push(inversePatches);
    this.redoStack = []; // Thao tác mới thì xóa nhánh redo
    if (this.undoStack.length > this.maxHistory) {
      this.undoStack.shift();
    }
  }

  // 2. Khi người dùng bấm Ctrl + Z
  undo(currentState: Manuscript): Manuscript | null {
    const inversePatches = this.undoStack.pop();
    if (!inversePatches) return null; // Hết cái để undo

    // Lưu lại patch xuôi để còn Redo được
    // (Trong thực tế bạn lưu cả cặp lúc record)
    return applyPatches(currentState, inversePatches);
  }

  // 3. Khi người dùng bấm Ctrl + Y
  redo(currentState: Manuscript): Manuscript | null {
    const patches = this.redoStack.pop();
    if (!patches) return null;

    return applyPatches(currentState, patches);
```

```
  }
}
```

Nhờ Immer đã làm hết phần khó nhất là tính toán chính xác mẩu dữ liệu nào bị thay đổi (tạo ra `inversePatches`), việc tự viết thêm hai mảng `undoStack` và `redoStack` trở nên đơn giản, hoạt động ổn định và kiểm soát được dung lượng RAM sử dụng.

User prompt: việc nhóm tự viết này có đề cập trong tài liệu không

Response: **Có, tài liệu đề xuất đã nêu rõ việc này.**

Tài liệu quy định việc tự triển khai cơ chế lịch sử thao tác (History / Undo / Redo) nằm trong gói `packages/editor-core` tại các mục sau:

1. Nơi chứa code trong cấu trúc Monorepo (Mục 2.3)

Tài liệu phân bổ một package riêng biệt cho tác vụ này:

- `packages/editor-core/`: phụ trách `store, commands, history, block registry`.
- File quản lý ngăn xếp Undo/Redo sẽ nằm trực tiếp trong thư mục con này (ví dụ: `editor-core/history.ts`).

2. Thiết kế Command + History theo hai Design Pattern kinh điển (Mục 6.2)

Mục **6.2 Command + History (Command, Memento)** mô tả chi tiết cách nhóm tự xây dựng cơ chế này:

- **Mẫu Command:** Mọi thay đổi dữ liệu phải đi qua `CommandBus` bằng hàm xử lý:

  ```
  // handler trả về: tài liệu mới (doc), danh sách patch xuôi và patch ngược
  handler: (doc, payload) => { doc: Manuscript; patches: Patch[]; inverse: Patch[] }
  ```

- **Mẫu Memento (Undo/Redo):** Tài liệu ghi rõ:

  > *"Lợi ích: undo/redo miễn phí (dùng patch ngược của Immer)... và mỗi command test được độc lập không cần React."*

  Cụ thể, `Immer` chỉ đóng vai trò cung cấp dữ liệu `inverse: Patch[]`, còn việc đón nhận mảng này, đưa vào danh sách lịch sử (ngăn xếp), và xử lý khi người dùng ấn Undo/Redo là code thuộc tầng điều phối `editor-core` do nhóm viết.

3. Phân công người phụ trách và lộ trình triển khai

- **Bảng Design Pattern (Mục 12):** Xác định **Command + Memento** là một trong ba pattern bắt buộc phải hoàn thành sớm nhất để phục vụ *"Mọi thao tác Editor, undo/redo, revision"*.
- **Lộ trình chuyển đổi (Mục 14 - Giai đoạn 4):** Đặt mốc bàn giao tính năng này:

  > *"Giai đoạn 4. Editor core: Store + CommandBus + history... Kết quả kiểm chứng: Undo/redo."*

- **Phân chia nhóm (Mục 15):** Nhóm **Editor** (phụ trách `editor-core`, `feature-*`) là đầu mối trực tiếp code phần quản lý lịch sử này.

User prompt: Persistence offline-first: IndexedDB (Dexie) đứng sau interface Repository; sau này Spring Boot/MySQL chỉ là thêm một adapter. còn ý này là gì vậy

Response: Ý này giải quyết bài toán: **Làm sao để người dùng không bao giờ bị mất bài khi rớt mạng, và sau này khi nhóm bắt tay vào làm Backend (Spring Boot + MySQL) thì không phải đập đi xây lại giao diện Editor.**

Để dễ hình dung, câu này gồm 3 ý chính sau:

1. "Persistence offline-first: IndexedDB (Dexie)" nghĩa là gì?

- **Persistence (Lưu trữ bền vững):** Dữ liệu không chỉ nằm tạm bợ trên RAM hay `sessionStorage` (tắt trình duyệt là mất sạch), mà được ghi thực sự xuống ổ cứng của máy tính.
- **Offline-first (Ưu tiên hoạt động ngoại tuyến):**
  - Bình thường, các web app truyền thống bắt buộc phải có mạng: bấm lưu là gửi API lên server, server lưu thành công thì mới báo xong. Nếu mạng chập chờn hoặc rớt mạng, người dùng không thể làm việc tiếp.

- *Offline-first* đảo ngược thứ tự: **Trình duyệt tự coi nó là một máy chủ thu nhỏ**. Mọi thao tác gõ chữ, lưu bài đều được ghi tức thì vào cơ sở dữ liệu nội bộ của trình duyệt (**IndexedDB**, thông qua thư viện hỗ trợ **Dexie**).
- Kể cả khi rút dây mạng, đi máy bay hay rớt Wi-Fi, bạn vẫn mở bài báo, chỉnh sửa, gõ công thức và tự động lưu bình thường 100%.

2. "Đứng sau interface Repository" nghĩa là gì?

Đây là việc áp dụng một mẫu thiết kế kinh điển: **Repository Pattern** (Mẫu kho lưu trữ).

- **Interface (Bản hợp đồng trừu tượng):** Editor không cần biết dữ liệu được lưu ở đâu (dưới ổ cứng máy tính hay trên đám mây). Editor chỉ biết nói chuyện với một "người thủ kho" thông qua một bản hợp đồng (`ManuscriptRepository`):

```
interface ManuscriptRepository {
  getById(id: string): Promise<Manuscript>;
  save(doc: Manuscript): Promise<void>;
  delete(id: string): Promise<void>;
}
```

- Màn hình soạn thảo (Editor) chỉ việc gọi lệnh đơn giản: `repo.save(bài_báo)`.
- Ở giai đoạn đầu, bạn lắp vào đó một "người thủ kho" chuyên cất đồ vào IndexedDB của máy tính (`IndexedDbManuscriptRepo`). Editor hoàn toàn không biết bên dưới đang dùng thư viện gì, chỉ cần biết bài đã được cất đi an toàn.

3. "Sau này Spring Boot/MySQL chỉ là thêm một adapter" nghĩa là gì?

- **Adapter (Đầu chuyển / Bộ chuyển đổi):** Giống như một cái đầu chuyển chân cắm sạc.
- Hiện tại, nhóm chưa kịp dựng xong Backend Spring Boot thì hệ thống vẫn chạy trơn tru cục bộ trên máy người dùng nhờ `IndexedDbManuscriptRepo`.
- Đến giai đoạn sau, khi nhóm đã viết xong API Spring Boot và tạo bảng MySQL:
  - Bạn **không cần phải sửa một dòng code nào trong màn hình Editor**.
  - Bạn chỉ việc viết thêm một class mới tên là `HttpManuscriptRepo` (hoặc `SyncedRepo`), bên trong chứa các lệnh gọi API (`fetch('/api/v1/manuscripts')`).
  - Sau đó chỉ cần "rút" ổ cắm IndexedDB cũ ra và "cắm" adapter mới vào. Editor vẫn gọi `repo.save(bài_báo)` như cũ, nhưng lần này dữ liệu sẽ tự động bay lên server Spring Boot và lưu vào cơ sở dữ liệu MySQL.

User prompt: tôi thấy trong ví dụ interface của bạn vừa rồi chỉ có phương thức thôi hả, còn thuộc tính đồ ko có lưu hả. Trong tài liệu có đề cập gì đến việc sẽ lưu cấu trúc bài báo ra sao trong indexdb ko ta, vì sau này cần lưu vào các bảng trong csdl mà

Response: 1. Tại sao `interface Repository` chỉ có phương thức (hàm) mà không có thuộc tính?

Trong thiết kế phần mềm chuẩn (Repository Pattern), **Repository không phải là nơi chứa dữ liệu, mà là "cửa ngõ" để thao tác với dữ liệu**.

- **Dữ liệu bài báo (có đầy đủ thuộc tính: tiêu đề, tác giả, nội dung...)** được định nghĩa riêng trong một kiểu dữ liệu khác gọi là Entity / Model: `interface Manuscript`.
- **Repository** đóng vai trò là một "người thủ kho" cung cấp các dịch vụ: nạp bài lên (`getById`), cất bài đi (`save`), xóa bài (`delete`). Nó không cần giữ các thuộc tính của bài báo bên trong bản thân nó, mà chỉ nhận đối tượng `Manuscript` vào làm tham số truyền vào hàm.

Sự tách bạch này giúp giao diện soạn thảo (Editor) không bị phụ thuộc vào việc dữ liệu được ghi vào đâu:

```
// 1. Đây là nơi chứa toàn bộ THUỘC TÍNH (Data Model)
interface Manuscript {
  id: string;
  title: string;
  body: Section[];
  updatedAt: string;
  // ...
}
```

```
// 2. Đây là CỬA NGÕ chỉ chứa PHƯƠNG THỨC (Repository Interface)
interface ManuscriptRepository {
  getById(id: string): Promise<Manuscript>;
  save(doc: Manuscript): Promise<void>;
  listRecent(): Promise<ManuscriptSummary[]>;
}
```

2. Tài liệu có đề cập cấu trúc lưu trữ bài báo trong IndexedDB không?

Có, tài liệu dành riêng Mục 9.2 để đặc tả chi tiết Lược đồ cơ sở dữ liệu IndexedDB (dùng thư viện Dexie).

Tài liệu không nhét nguyên một cục JSON khổng lồ vào một chỗ, mà chia thành **8 bảng chuyên biệt** trong IndexedDB của trình duyệt:

Tên bảng trong IndexedDB	Khóa chính / Chỉ mục (Index)	Nội dung lưu trữ thực tế
`manuscripts`	`id`, index `[journalId+updatedAt]`, `status`	Toàn bộ cây bài báo (`Manuscript` JSON) **nhưng đã tách toàn bộ ảnh ra ngoài**; lưu kèm số phiên bản `rev`, thời gian sửa đổi.
`revisions`	`id`, index `[manuscriptId+rev]`	Chuỗi các thao tác thay đổi (chuỗi Patch sinh ra từ Immer) giữa các lần sửa để phục vụ lịch sử chỉnh sửa và khôi phục khi cần.
`assets`	`hash` (mã băm sha256)	Dữ liệu nhị phân (Blob) của từng bức ảnh, kèm chiều rộng, chiều cao, DPI, định dạng file. Tránh lưu ảnh dạng base64 làm phình dữ liệu.
`assetRefs`	`[manuscriptId+hash]`	Bảng nối để biết bài báo nào đang dùng bức ảnh nào (dùng để dọn dẹp các bức ảnh bị xóa khỏi bài).
`ingestSources`	`manuscriptId`	File Word gốc (`.docx`) kèm theo báo cáo bóc tách lỗi ban đầu.
`outbox`	`seq`, `manuscriptId`	Hàng đợi các thao tác đang chờ gửi lên server khi có mạng lại.
`compileCache`	`hash`	Lưu sẵn file PDF đã biên dịch để không phải gọi server biên dịch lại nếu nội dung không đổi.
`settings`	`key`	Cấu hình giao diện và lựa chọn người dùng.

3. Lưu dạng JSON trên trình duyệt thì sau này chuyển lên các bảng MySQL thế nào?

Tài liệu đã tính trước bài toán này ở **Mục 10.2 (Lược đồ MySQL phía Spring Boot)** theo phương pháp thiết kế **Aggregate (Cụm thực thể)** của kiến trúc hướng miền (DDD):

Tại sao bài báo không bị xé vụn ra hàng chục bảng quan hệ (RDBMS)?

Một bài báo khoa học có cấu trúc cây lồng nhau rất sâu: một bài có nhiều chương, trong chương có nhiều đoạn, trong đoạn có bảng/hình/công thức. Nếu xé nhỏ từng đoạn văn, từng công thức thành các dòng riêng trong các bảng `sections`, `paragraphs`, `equations` của MySQL:

- Mỗi lần mở một bài báo, Spring Boot phải chạy hàng chục lệnh `JOIN` phức tạp, làm chậm hệ thống và rất dễ lỗi khi cập nhật thứ tự các khối.

Giải pháp tài liệu đề xuất: Hybrid Storage (Vừa quan hệ vừa JSON)

Hệ thống tận dụng tính năng **Native JSON của MySQL 8**:

- Toàn bộ cây nội dung chi tiết của bài báo được lưu nguyên vẹn vào một cột kiểu `JSON` mang tên `doc_json` trong bảng `manuscripts`.
- Các thông tin cần thiết để **tìm kiếm, phân quyền và lọc danh sách** được tách ra thành các cột quan hệ thông thường bên ngoài:

```
CREATE TABLE manuscripts (
    id VARCHAR(26) PRIMARY KEY,      -- Mã ULID
    owner_id BIGINT NOT NULL,        -- Tác giả sở hữu (Foreign key tới bảng users)
```

```
    journal_id VARCHAR(50) NOT NULL, -- Tạp chí nộp vào
    title TEXT,                      -- Tách riêng để tìm kiếm nhanh
    doi VARCHAR(100),                -- Mã DOI
    status VARCHAR(30),              -- Trạng thái: DRAFT, SUBMITTED...
    rev INT NOT NULL DEFAULT 1,      -- Số phiên bản (kiểm soát khóa lạc quan @Version)
    doc_json JSON NOT NULL,          -- TOÀN BỘ CÂY MANUSCRIPT V2 LƯU Ở ĐÂY
    updated_at TIMESTAMP
);
```

Nhờ thiết kế này:

1. **Dữ liệu khớp 1:1 giữa hai phía:** Cấu trúc JSON dưới `IndexedDB` của trình duyệt khi gửi lên API sẽ được nạp thẳng vào cột `doc_json` của MySQL mà không cần qua các bước chuyển đổi (mapping) phức tạp.
2. **Ảnh lưu độc lập:** Cả trên IndexedDB lẫn trên MySQL, ảnh đều được lưu riêng theo mã băm (`sha256`) và cất vào kho lưu trữ file (S3/MinIO hoặc thư mục máy chủ), không làm nặng bảng dữ liệu.

User prompt: ủa mà minIO là gì vậy

Response: **MinIO** là một phần mềm mã nguồn mở chuyên dùng để **lưu trữ tập tin (Object Storage)**, đóng vai trò như một phiên bản tự cài đặt (self-hosted) của dịch vụ **Amazon S3 (AWS S3)**.

1. Tại sao lại dùng MinIO thay vì lưu thẳng vào MySQL hay ổ cứng server?

Trong các bài báo khoa học, người dùng tải lên rất nhiều file nặng: ảnh quét kính hiển vi chất lượng cao, đồ thị vector, file PDF thành phẩm, file Word gốc.

- **Nếu lưu thẳng vào cơ sở dữ liệu MySQL (dưới dạng BLOB/Base64):** Cơ sở dữ liệu sẽ bị phình to cực nhanh từ vài chục MB lên hàng trăm GB. Khi dung lượng quá lớn, việc sao lưu (backup), đánh chỉ mục (index) và truy vấn dữ liệu dạng chữ sẽ bị chậm chạp rõ rệt.
- **Nếu lưu thẳng vào một thư mục trên ổ cứng máy chủ chạy Spring Boot:** Khi hệ thống mở rộng, nếu bạn bật 2–3 máy chủ Spring Boot chạy song song để cân bằng tải, máy chủ A sẽ không nhìn thấy ảnh được lưu trên ổ cứng của máy chủ B.
- **Giải pháp với MinIO:**
  - Spring Boot chỉ lưu **đường link hoặc mã băm (`sha256`)** của bức ảnh trong MySQL.
  - Toàn bộ dữ liệu file ảnh, PDF gốc được gửi sang lưu tập trung tại MinIO.
  - MinIO chịu trách nhiệm quản lý dung lượng, phân phối file và truyền tải dữ liệu nhanh qua giao thức HTTP/HTTPS.

2. Ưu điểm cốt lõi của MinIO trong dự án

- **Tương thích 100% với chuẩn Amazon S3 (S3-compatible):** Code Java/Spring Boot dùng thư viện chính thức `aws-java-sdk-s3` để giao tiếp với MinIO. Sau này nếu trường học hoặc tòa soạn chuyển hệ thống lên đám mây AWS, nhóm chỉ cần đổi đúng 1 dòng URL cấu hình sang AWS S3 mà **không phải sửa một dòng code Java nào**.
- **Dễ dàng chạy thử nghiệm cục bộ (Local / Dev):** Bạn có thể dựng một server MinIO hoàn chỉnh trên máy tính cá nhân chỉ bằng một dòng lệnh Docker:

  ```
  docker run -p 9000:9000 -p 9001:9001 minio/minio server /data --console-address ":9001"
  ```

- **Có giao diện trực quan (Web Console):** MinIO đi kèm giao diện web giúp bạn đăng nhập bằng trình duyệt để xem các thư mục (bucket), kiểm tra ảnh tải lên, tải file về và phân quyền một cách trực quan.

User prompt: Compile service tách riêng, sandbox, có cache/queue/cancel; dùng chung cho PDF preview và PDF xuất bản (PDF/X). còn ý này là sao vậy

Response: Ý này giải quyết bài toán cốt lõi: **Làm sao để biên dịch file LaTeX ra PDF an toàn, nhanh, không làm sập máy chủ web và đảm bảo chất lượng in ấn chuyên nghiệp.**

Để dễ hình dung, câu này được bóc tách thành 4 phần kỹ thuật:

1. Compile service tách riêng (Dedicated Compiler Service)

- **Bản cũ (v1) bị gì?** Biên dịch LaTeX đang bị làm ở 2 nơi tùy tiện: vừa nằm trong route của Next.js (Node.js), vừa nằm trong code Spring Boot. Chạy lệnh hệ thống (`child_process.exec('xelatex...')`) trực tiếp ngay trên máy chủ web sẽ khiến máy chủ dễ bị treo đứng khi có vài người cùng bấm xem PDF.
- **Bản mới (v2) làm gì?** Rút toàn bộ môi trường TeX Live, font chữ và các thư viện biên dịch ra thành **một dịch vụ độc lập** chạy trong một Docker container riêng (`services/compiler-worker`). Ứng dụng web (Next.js/Spring Boot) chỉ gửi yêu cầu sang worker này xử lý, không bao giờ tự chạy tiến trình LaTeX trên web server.

2. Sandbox (Hộp cát cách ly an toàn tuyệt đối)

Mã LaTeX thực chất là một ngôn ngữ lập trình hoàn chỉnh. Nếu người dùng dán vào bài viết các đoạn mã độc như:

```
\immediate\write18{rm -rf /}      % Lệnh xóa sạch ổ cứng máy chủ Linux
\input{/etc/passwd}               % Lệnh đọc trộm mật khẩu hệ thống
```

Trình biên dịch mặc định sẽ thực thi các lệnh đó ngay lập tức. Vì vậy, **Sandbox** là việc nhốt trình biên dịch vào một "lồng bảo vệ":

- Chạy với tùy chọn cấm thực thi lệnh hệ thống: `--no-shell-escape`.
- Chặn không cho đọc/ghi file bên ngoài thư mục bài viết: `openin_any=p`, `openout_any=p`.
- Container biên dịch bị ngắt kết nối Internet hoàn toàn, chạy dưới quyền tài khoản không có quyền root (`non-root user`), đồng thời bị giới hạn nghiêm ngặt về RAM và thời gian chạy tối đa (timeout).

3. Có cache / queue / cancel (Quản lý hàng đợi và hiệu năng thông minh)

- **Cache (Bộ nhớ đệm theo mã băm):** Khi người dùng bấm biên dịch, hệ thống tạo mã băm `hash(nội dung tex + danh sách ảnh + profile tạp chí)`. Nếu mã hash này trùng khớp với lần chạy trước, worker trả về ngay file PDF đã có trong bộ nhớ mà **không cần chạy lại XeLaTeX**.
- **Queue (Hàng đợi điều phối):** Khi có hàng chục tác giả cùng ấn xuất bản, worker không chạy ồ ạt làm cháy CPU mà xếp các yêu cầu vào một hàng đợi (Queue), xử lý tuần tự theo khả năng của máy chủ.
- **Cancel (Hủy tác vụ cũ):** Khi một người dùng gõ sửa liên tục và ấn nút biên dịch 3 lần liên tiếp, worker tự động **hủy (kill) ngay 2 tác vụ trước** đang chạy dở để dồn tài nguyên cho lần biên dịch mới nhất, tránh tình trạng hàng đợi bị nghẽn vô ích.

4. Dùng chung cho PDF preview và PDF xuất bản (PDF/X)

Hệ thống sử dụng **cùng một dây chuyền biên dịch duy nhất** cho cả hai mục đích, nhưng linh hoạt ở bước hoàn thiện cuối:

Mục đích	Đầu ra	Quy trình xử lý của Worker
PDF Preview (Xem trước lúc soạn thảo)	PDF tiêu chuẩn, nhẹ, nét	`xelatex` chạy nhanh → trả về PDF tức thì để người dùng xem vị trí ngắt dòng, ngắt trang.
PDF xuất bản (Bản in công bố nhà in)	**PDF/X** hoặc **PDF/A** (chuẩn in ấn thương mại và lưu trữ)	Sau khi `xelatex` xuất xong, file sẽ đi tiếp qua công cụ **Ghostscript / Prepress**: nhúng 100% font chữ, chuyển đổi hệ màu CMYK/Grayscale chuẩn in, kiểm tra ảnh đạt tối thiểu 300 DPI, gắn metadata bản quyền và tạo mục lục điện tử (bookmarks).

Cách làm này đảm bảo những gì tác giả nhìn thấy trên màn hình xem trước (Preview) khớp chính xác với bản in thực tế của tòa soạn (PDF/X).

User prompt: nãy giờ còn cái ý nào trong tài liệu đề xuất mà tôi chưa hỏi bạn không

Response: Có. Xuyên suốt từ đầu buổi đến giờ, bạn đã hỏi rất kỹ về:

1. **Mô hình lõi (`doc-model`, AST, ID ổn định, Derived data)**
2. **Cơ chế sinh mã và tối ưu biên dịch LaTeX (`.cls`, AST → TeX, kỹ thuật tăng tốc)**
3. **Pipeline đọc file Word (`ingest-docx`, Web Worker, Isomorphic)**
4. **Cơ chế cộng tác chia việc và cấu trúc Block Registry**
5. **Cấu hình tạp chí (`Journal Profile`)**

6. **Kiến trúc Editor (Store, Command, Immer, Undo/Redo)**
7. **Lưu trữ ngoại tuyến và cơ sở dữ liệu (`IndexedDB`, `MySQL`, `MinIO`, `Repository Pattern`)**
8. **Dịch vụ biên dịch (`compiler-worker`, Sandbox, PDF/X)**

Tuy nhiên, tài liệu đề xuất vẫn còn **6 cụm vấn đề quan trọng** dưới đây mà bạn chưa đề cập tới:

1. Động cơ kiểm tra chất lượng tự động (QC / Preflight Engine — Mục 7)

- **Ý nghĩa:** Trước khi cho phép xuất bản hoặc nộp bài, hệ thống chạy một loạt luật kiểm tra tự động (**Rule Engine** theo mẫu *Specification Pattern*).
- **Các lỗi bắt tự động:**
  - Trích dẫn tài liệu tham khảo ma (`[5]` nhưng danh mục chỉ có 4 tài liệu, hoặc tài liệu số 3 không bao giờ được nhắc đến trong bài).
  - Hình ảnh/bảng biểu không được nhắc tới trước khi xuất hiện, hoặc nhắc tới hình không tồn tại.
  - Ảnh độ phân giải thấp (dưới 300 DPI, bị vỡ khi in ấn).
  - Đối chiếu DOI với Volume/Issue (phát hiện ra lỗi có thật trong bài mẫu ID630_HS).
  - Báo lỗi tràn chữ (`Overfull \hbox`), font chữ chưa được nhúng vào PDF.
- **Nút "Sửa nhanh" (Quick Fix):** Mỗi lỗi phát hiện ra có thể gắn kèm một `Command`, cho phép người dùng bấm một nút là hệ thống tự sửa lỗi ngay trên Editor.

2. Các bộ xuất dữ liệu khác ngoài LaTeX (JATS XML, DOCX, BibTeX — Mục 8.1)

- Bạn đã hỏi rất kỹ về việc xuất LaTeX, nhưng tài liệu còn quy định xuất ra:
  - **JATS XML (Journal Article Tag Suite):** Chuẩn XML quốc tế bắt buộc để bài báo được chỉ mục lên các cơ sở dữ liệu học thuật toàn cầu (như PubMed, Crossref, Scopus). Nhờ cấu trúc cây AST của `Manuscript v2`, việc xuất sang JATS XML gần như có sẵn.
  - **Xuất ngược lại Word (`.docx`):** Giữ đúng bộ style của `Template_EN/VI` để trả lại bản thảo cho tác giả nếu cần phản biện kín.
  - **Đóng gói file `.zip`:** Gom toàn bộ file `.tex`, `.bib` và thư mục ảnh chuẩn hóa để nộp lên tòa soạn hoặc import thẳng vào Overleaf.

3. Tích hợp AI có kiểm soát và bảo mật bản thảo (Mục 11)

- **Khắc phục lỗi bảo mật:** Bản v1 để lộ `OPENAI_API_KEY` ngay trên mã nguồn frontend và bắt người dùng gõ `sk-...`.
- **Cơ chế mới:**
  - Khóa API nằm hoàn toàn ở Backend proxy.
  - **Không gửi cả bài báo lên AI** (tránh lộ công trình nghiên cứu chưa công bố và tốn token). Chỉ gửi những đoạn nhỏ mà bộ bóc tách Word đánh giá là mơ hồ (độ tin cậy thấp).
  - **AI chỉ là người đề xuất, không ghi đè:** Kết quả AI trả về hiển thị dưới dạng gợi ý (gạch đỏ/xanh) để tác giả bấm "Chấp nhận" hoặc "Từ chối", không tự ý thay đổi dữ liệu thật.

4. Chiến lược xử lý Typography chuyên sâu (Mục 5.3)

- **Các quy tắc in ấn học thuật:**
  - **Ngắt từ tự động (Hyphenation):** Tiếng Việt chủ yếu là từ đơn âm tiết nên mặc định tắt để tránh ngắt đôi từ vô nghĩa; tiếng Anh thì bật.
  - **Chống dòng mồ côi / góa bụa (Widow/Orphan):** Tự động thêm `\widowpenalty`, `\clubpenalty` để tránh việc chỉ có một dòng chữ trơ trọi nằm ở đầu hoặc cuối trang.
  - **Khoảng cách ký tự (Tracking/Letterspacing):** Cấu hình qua package `microtype` để chữ dàn đều hai cột A4 mà không bị khoảng trắng lỗ chỗ.

5. Chiến lược kiểm thử tự động (Testing & Golden Tests — Mục 13)

- **Golden Tests (Kiểm thử chuẩn vàng):** Dùng các file `.docx` mẫu và đối chiếu với file JSON chuẩn để đảm bảo sửa code parser không bao giờ làm hỏng các bài báo cũ.
- **Visual Regression Testing:** Biên dịch file LaTeX ra PDF, chụp ảnh từng trang rồi so sánh từng pixel (`pixelmatch`) với file PDF mẫu ID630_HS để bắt ngay lập tức hiện tượng lệch dòng/lệch cột khi ai đó sửa file `.cls`.
- **Kiểm soát ranh giới tầng:** Dùng `dependency-cruiser` và `ArchUnit` trong CI để chặn lập trình viên import chéo (ví dụ: cấm `doc-model` import React).

6. Năm câu hỏi cốt lõi cần đội ngũ đưa ra quyết định (Mục 17)

Tài liệu kết thúc bằng 5 câu hỏi lớn để quyết định phạm vi triển khai:

1. Có làm tính năng nhiều người cùng sửa bài theo thời gian thực (như Google Docs) không, hay chỉ cần lưu tuần tự theo phiên bản?
2. Chuẩn PDF/X và hỗ trợ người khiếm thị đọc PDF có bắt buộc ngay từ đầu không (để quyết định dùng XeLaTeX hay chuyển sang LuaLaTeX)?
3. Font thương mại (Minion Pro, Myriad Pro) có mua bản quyền trên server không, hay dùng font nguồn mở thay thế?
4. Định dạng trích dẫn chuẩn của từng tạp chí là gì (dấu ngoặc vuông `[1]` hay chỉ số trên `$^{(1)}$`)?
5. Mức độ bắt buộc tác giả dùng Template: Bắt buộc 100% dùng đúng template Word chuẩn, hay vẫn phải cho phép nộp file Word định dạng tự do?

User prompt: nãy tôi thấy có một tính năng là đặt một thẻ ẩn gì đó trong file latex để khi nó biên dịch ra thì nhấn vào preview sẽ dẫn ngay đến chỗ chính xác trong giao diện để sửa á. Tôi hiểu như vậy có đúng không 2. chuẩn pdf/x là gì vậy

Response: 1. Bạn hiểu như vậy có đúng không?

Bạn hiểu hoàn toàn chính xác về mặt mục đích, nhưng về cơ chế kỹ thuật thì có hai chiều liên kết cần phân biệt rõ:

Trong tài liệu đề xuất (Mục 6.5), tính năng này kết hợp **2 cơ chế** để hỗ trợ tương tác hai chiều giữa giao diện soạn thảo (Editor) và bản PDF:

Chiều 1: Nhấn vào thông báo lỗi → Nhảy đến đúng khối soạn thảo (Dùng thẻ ẩn `%%BLOCK:<id>`)

- **Cách hoạt động:** Khi hệ thống sinh mã LaTeX từ cây dữ liệu, trước mỗi đoạn văn, bảng hay hình ảnh, bộ xuất dữ liệu sẽ chèn một dòng chú thích ẩn không ảnh hưởng đến nội dung in ra, ví dụ:

```
%%BLOCK:01JC8Z4K5M9PQR3VWT7X2B1D6Y
\begin{equation}
  x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}
\end{equation}
```

- **Khi biên dịch bị lỗi:** Trình biên dịch LaTeX (XeLaTeX) sẽ xuất ra file log: *"Lỗi cú pháp tại dòng 142"*.
- Hệ thống sẽ đọc ngược từ dòng 142 lên trên để tìm thẻ `%%BLOCK:<id>` gần nhất. Nhờ đó, trên giao diện báo lỗi, người dùng chỉ cần **bấm vào dòng báo lỗi là con trỏ tự động nhảy đến đúng khối công thức đó** trên màn hình soạn thảo để sửa, thay vì phải tự dò tìm trong bài.

Chiều 2: Bấm vào một vị trí trên trang PDF → Nhảy đến đúng chữ trong Editor (Dùng SyncTeX)

- **SyncTeX (Synchronize TeX n' PDF):** Đây là một công nghệ tích hợp sẵn của hệ sinh thái TeX. Khi biên dịch với cờ `-synctex=1`, trình biên dịch sẽ tạo ra một file bản đồ tọa độ (`article.synctex.gz`).
- File này lưu trữ chính xác mối liên hệ: *Tọa độ (X, Y) trên trang 3 của PDF tương ứng với dòng mấy, ký tự mấy trong file mã nguồn*.
- Khi bạn dùng chuột click đúp vào một đoạn văn bất kỳ trên khung xem trước PDF, trình đọc PDF (`pdf.js`) kết hợp với dữ liệu SyncTeX sẽ lập tức cuộn màn hình Editor đến đúng vị trí đoạn văn đó.

2. Chuẩn PDF/X là gì?

PDF/X (PDF for eXchange) là một tiêu chuẩn quốc tế (ISO) chuyên biệt dành riêng cho **ngành in ấn thương mại và chế bản xuất bản (Prepress)**.

Nói một cách dễ hiểu:

- Một file PDF thông thường bạn mở trên máy tính thấy rất đẹp, nhưng khi mang ra nhà in công nghiệp để in hàng nghìn bản tạp chí thì có thể bị: **mất chữ, nhảy font, sai màu, hoặc hình ảnh in ra bị mờ căm**.
- **PDF/X sinh ra để triệt tiêu mọi rủi ro đó**, với nguyên tắc: *"Những gì hiển thị trên file số chắc chắn sẽ in ra giống hệt 100% trên giấy in ấn thực tế"*.

Để đạt chuẩn PDF/X, file xuất ra phải vượt qua các điều kiện khắt khe:

1. **Bắt buộc phải nhúng 100% Font chữ (Font Embedding):**
    - File PDF thông thường có thể chỉ "mượn tạm" font cài sẵn trên máy bạn. Nếu máy in của tòa soạn thiếu font đó, nó sẽ tự thế bằng font khác → vỡ bố cục hoặc lỗi font tiếng Việt.
    - PDF/X bắt buộc nhúng toàn bộ dữ liệu font vào thẳng bên trong file PDF. Máy in mở lên là có sẵn font để in.
2. **Quản lý hệ màu in ấn chuẩn (CMYK / Grayscale):**
    - Màn hình máy tính hiển thị hệ màu **RGB** (ánh sáng), còn máy in dùng hệ 4 màu mực **CMYK** (mực in).
    - PDF/X buộc phải khai báo cấu hình màu máy in đích (Color Profile), chuyển đổi màu sắc chính xác để bản in không bị xỉn màu hay lệch tông.
3. **Độ phân giải hình ảnh tối thiểu (DPI):**
    - Màn hình chỉ cần 72–96 DPI là nét, nhưng bản in học thuật đòi hỏi tối thiểu **300 DPI**. PDF/X sẽ từ chối các file có ảnh độ phân giải thấp gây vỡ hạt khi in.
4. **Cấm hoàn toàn các nội dung động:**
    - Cấm âm thanh, video, mã JavaScript chạy ngầm, liên kết ngoài nguy hiểm hoặc các lớp form tương tác. File phải hoàn toàn tĩnh và bất biến.

Trong kiến trúc của StructPaper v2 (Mục 8.2), sau khi XeLaTeX xuất ra PDF thô, file sẽ được chuyển qua công cụ **Ghostscript** để "ép chuẩn" thành **PDF/X**, đảm bảo file tải về sẵn sàng gửi thẳng tới xưởng in của Đại học Quốc gia mà không cần kỹ thuật viên nhà in phải chỉnh sửa thủ công.

User prompt: chiều 2 đó trong tài liệu có đề cập đến việc triển khai nó hay không, và nhờ cơ chế nào mà nó biết chính xác cái nào trong editor vậy nhớ cái ghi chú ở chiều 1 á hả 2. chiều 1 có thể nhảy đúng chỗ soạn thảo thì liệu là chiều 1 có thể làm kiểu bấm vào lỗi thì preview pdf cuộn đến đũng chỗ lỗi đó luôn không

Response: 1. Chiều 2 (Bấm vào PDF → Nhảy đến Editor)

Tài liệu có đề cập đến việc triển khai không?

Có. Tài liệu đề cập trực tiếp ở **Mục 6.5 (Preview hai tầng)** với nội dung:

> *"Từ khi emitter chèn comment `%%BLOCK:<id>` vào TeX, có thể ánh xạ ngược lỗi LaTeX → block (bấm lỗi để nhảy tới đúng đoạn) và dùng SyncTeX cho bấm vào PDF → nhảy tới block."*

Tuy nhiên, tài liệu chỉ nêu ra **hướng giải pháp (quyết định kiến trúc)** chứ chưa viết chi tiết từng hàm thực thi; việc lập trình cụ thể thuộc về gói `editor-core` và `feature-preview`.

Nhờ cơ chế nào mà hệ thống biết chính xác vị trí trong Editor? Có phải nhờ ghi chú `%%BLOCK:<id>` không?

Đúng, nhưng là sự kết hợp chặt chẽ giữa 2 chiếc cầu nối:

Nếu chỉ có SyncTeX thì chưa đủ, vì SyncTeX vốn chỉ ánh xạ giữa PDF và **dòng/cột trong file text** `.tex`, trong khi Editor của bạn lại quản lý theo **cây Block và ô nhập liệu**.

Quy trình giải mã vị trí diễn ra qua 3 bước:

```
[Click chuột trên PDF (Trang p, tọa độ X, Y)]
                      |
                      ▼ (Cầu nối 1: SyncTeX)
[Dòng số L trong file article.tex]
                      |
                      ▼ (Cầu nối 2: Thẻ ghi chú %%BLOCK:<id>)
[ID của khối dữ liệu (blockId)]
                      |
                      ▼ (Editor DOM / React Ref)
[Cuộn màn hình đến Block đó & Focus con trỏ vào ô nhập]
```

1. **Bước 1 (PDF → Dòng TeX):** Khi bạn bấm chuột vào trang PDF (đang hiển thị bằng thư viện `pdf.js`), tọa độ click (x, y, page) được gửi tới thư viện đọc dữ liệu SyncTeX (như `synctex-js`). SyncTeX tra bảng tọa độ và trả về: *Vị trí này được sinh ra từ dòng số L trong file `article.tex`*.

2. **Bước 2 (Dòng TeX → Block ID):** Bộ sinh LaTeX (`LatexExporter`) khi tạo file `.tex` đã lưu sẵn một bản đồ chỉ mục: dòng L này đang nằm ngay dưới thẻ `%%BLOCK:<id>` nào. Từ đó, hệ thống xác định được chính xác `blockId`.

3. **Bước 3 (Block ID → Giao diện):** Trên màn hình Editor, mỗi khối đều mang thuộc tính HTML dạng `data-block-id="<id>"`. Hệ thống chỉ cần gọi lệnh:

   ```
   document.querySelector(`[data-block-id="${blockId}"]`)?.scrollIntoView({ behavior: 'smooth' });
   ```

   Màn hình soạn thảo sẽ tự động cuộn đến đúng khối đó và đặt con trỏ chuột vào vị trí cần sửa.

2. Bấm vào lỗi biên dịch → PDF có cuộn đến đúng chỗ lỗi được không?

Về mặt kỹ thuật: Thường là KHÔNG thể (hoặc rất hạn chế), vì bản chất lỗi biên dịch làm gãy chuỗi sinh PDF.

Cơ chế này gặp phải 2 rào cản kỹ thuật thực tế sau:

Rào cản 1: Lỗi biên dịch đồng nghĩa với việc "chưa có trang PDF tương ứng"

Khi XeLaTeX gặp lỗi cú pháp nghiêm trọng (Fatal Error — ví dụ: công thức toán thiếu dấu đóng ngoặc `}`, bảng thừa cột, thẻ môi trường chưa đóng `\end{...}`):

- Tiến trình biên dịch sẽ **dừng lại ngay lập tức** hoặc thoát với mã lỗi.
- Toàn bộ các trang từ vị trí lỗi trở về sau **chưa hề được dàn trang và chưa xuất hiện trong file PDF**.
- Vì phần bị lỗi chưa tồn tại trên PDF nên PDF Viewer không có tọa độ hay trang nào để cuộn tới.

Rào cản 2: SyncTeX không ghi nhận các khối bị lỗi

SyncTeX chỉ ghi nhận tọa độ của những phần tử đã được dàn trang thành công vào các "hộp chữ" (TeX boxes). Một khối lệnh gây sập trình biên dịch thì chưa được đưa vào hộp chữ nào, do đó file `.synctex` hoàn toàn không có tọa độ cho đoạn đó.

Hướng xử lý thực tế tốt nhất cho trường hợp này

Thay vì cố cuộn một trang PDF chưa được tạo ra, luồng trải nghiệm chuẩn (đúng như Mục 6.5 và 7 trong tài liệu) sẽ là:

1. **Bấm vào dòng lỗi → Màn hình Editor (bên trái) cuộn tới đúng khối bị lỗi:** Hệ thống dùng thẻ `%%BLOCK:<id>` để tìm ra khối bị sai (ví dụ: khối công thức toán `eq-02`), đánh dấu viền đỏ và mở modal sửa nhanh.

2. **Trên khung PDF Preview (bên phải):** Giữ nguyên trang PDF của lần biên dịch thành công gần nhất và hiển thị một thanh thông báo nhỏ (banner):

   > *"Biên dịch thất bại tại Khối Công thức 2. Đang hiển thị bản xem trước của lần chạy trước."*

3. **Trường hợp duy nhất cuộn được PDF:** Nếu đó chỉ là **Cảnh báo (Warning)** như chữ tràn lề (`Overfull \hbox`) hoặc ảnh mờ DPI thấp (PDF vẫn sinh ra thành công). Lúc này, hệ thống lấy dòng cảnh báo trong log → dùng SyncTeX chiều thuận (Forward Search: từ dòng TeX tìm ra tọa độ trang PDF) → cuộn PDF đến đúng dòng bị tràn chữ và tô vàng vị trí đó.

User prompt: ko ,ý là lỗi nhỏ như là vượt max từ cho khối tóm tắt chẳng hạn, ví dụ vậy á

Response: Với các lỗi nội dung hoặc quy định tạp chí như **vượt số từ tóm tắt, thiếu dòng nguồn của bảng, ảnh bị mờ DPI thấp**, câu trả lời là: **Hoàn toàn làm được và làm rất mượt mà.**

Những lỗi này không làm gãy trình biên dịch. XeLaTeX vẫn chạy thành công 100% và file PDF vẫn được sinh ra đầy đủ.

Cơ chế hoạt động: Dùng "Tìm kiếm xuôi" (Forward Search) của SyncTeX

Khi biên dịch thành công, luồng điều hướng từ danh sách lỗi sang PDF diễn ra qua 4 bước:

```
[Bấm vào lỗi: "Tóm tắt vượt quá 500 từ"]
                    │
                    ▼
1. Lấy blockId của khối Tóm tắt (từ đối tượng Issue trong QC Engine)
                    │
                    ▼
2. Tra bản đồ chỉ mục: blockId này nằm ở dòng mấy trong file .tex
                    │
                    ▼
3. Gọi SyncTeX chiều xuôi (Forward Search): Dòng L trong .tex ➔ Nằm ở trang mấy, tọa độ (X, Y) nào trên PDF
                    │
                    ▼
4. pdf.js tự cuộn màn hình xem trước đến đúng trang đó và đánh dấu vị trí
```

1. **Bước 1 (Xác định khối):** Khi hệ thống kiểm tra chất lượng (QC Engine) phát hiện đoạn tóm tắt có 550 từ trong khi tạp chí chỉ cho phép tối đa 500 từ, nó tạo ra một thông báo lỗi có gắn mã định danh:

   ```
   {
     ruleId: 'abstract.maxWords',
     message: 'Tóm tắt đang có 550 từ (tối đa 500 từ)',
     target: { blockId: 'block-abstract-01' }
   }
   ```

2. **Bước 2 (Tìm dòng mã TeX tương ứng):** Khi `LatexExporter` sinh file `.tex`, nó đã ghi nhớ khối `block-abstract-01` nằm từ dòng 45 đến dòng 60 (ngay dưới thẻ `%%BLOCK:block-abstract-01`).

3. **Bước 3 (Tra tọa độ PDF qua SyncTeX):** Hệ thống nạp số dòng 45 vào thư viện SyncTeX theo chiều **Forward Search** (từ mã nguồn TeX sang trang PDF). SyncTeX trả về kết quả: *Dòng này nằm ở Trang 1, vùng tọa độ (x: 50, y: 320, width: 500, height: 120)*.

4. **Bước 4 (Cuộn và làm nổi bật):** Khung xem trước PDF (`pdf.js`) nhận lệnh:
   - Cuộn thẳng đến Trang 1.
   - Vẽ một khung viền đỏ/vàng nhấp nháy đè lên khối Tóm tắt trên trang PDF để mắt người dùng bắt ngay được điểm cần chú ý.

Trải nghiệm đồng bộ cả hai màn hình (Side-by-Side)

Khi kết hợp cơ chế này, bạn sẽ có một luồng tương tác:

- Bấm vào dòng lỗi trong danh sách QC:
  - **Nửa màn hình bên trái (Editor):** Tự cuộn đến ô nhập Tóm tắt, bôi đỏ các từ thừa và đặt con trỏ chuột sẵn để bạn gõ xóa bớt.
  - **Nửa màn hình bên phải (PDF Preview):** Đồng thời tự cuộn đến khối Tóm tắt trên bản in PDF để bạn thấy khung chữ đó thực tế đang chiếm diện tích lớn thế nào trên trang báo.

User prompt: thao tác này có diễn ra nhanh được hông á

Response: Rất nhanh, thao tác này diễn ra gần như **tức thì (khoảng 30 – 50 mili-giây)**, bạn vừa bấm chuột là cả hai bên màn hình cuộn ngay lập tức mà không có độ trễ cảm nhận được.

Sở dĩ thao tác này đạt tốc độ thời gian thực là vì:

- **Không phải chạy lại LaTeX:** Khi danh sách lỗi đã hiện lên, file PDF và file tọa độ SyncTeX đều đã được sinh ra và nạp sẵn trong bộ nhớ từ trước. Bấm vào lỗi chỉ là việc tra cứu dữ liệu có sẵn chứ không gọi tiến trình biên dịch nào cả.

- **Tra cứu tọa độ siêu nhẹ:**
  - Tra từ `blockId` sang dòng mã TeX: Chỉ là một phép tra bảng băm (Map lookup), tốn chưa tới **1ms**.
  - Tra từ dòng TeX sang tọa độ (x, y, trang) qua SyncTeX: Thực thi bằng thuật toán tìm kiếm nhị phân (Binary Search) trên mảng tọa độ trong RAM, chỉ tốn khoảng **2 – 5ms**.
- **Thao tác cuộn chỉ là xử lý giao diện thuần túy:** Lệnh cuộn khung soạn thảo và chuyển trang trên `pdf.js` tốn khoảng **16ms** (bằng đúng thời gian quét một khung hình 60fps của màn hình).

Hai lưu ý nhỏ khi triển khai để giữ tốc độ tối đa:

1. **Phân tích SyncTeX từ sớm:** Ngay khi máy chủ trả về file PDF kèm `article.synctex.gz`, hãy giải nén và chuyển cấu trúc tọa độ này vào RAM trình duyệt luôn. Tránh việc đợi người dùng bấm vào dòng lỗi rồi mới bắt đầu giải nén file `.gz`.
2. **Vẽ khung viền nổi bật bằng thẻ HTML đè lên (Overlay):** Tạo một thẻ `<div>` trong suốt có viền đỏ nằm đè lên vị trí (x, y) của PDF thay vì yêu cầu `pdf.js` vẽ lại toàn bộ trang lên Canvas, vừa nhẹ máy vừa không bị giật khung hình.

User prompt: cái bản này có update một số điều về định dạng reference so với bản trước á, chúng là gì vậy

Response: Trong bản đề xuất v2, toàn bộ cách quản lý trích dẫn và tài liệu tham khảo (Citation & Reference) được tái cấu trúc từ gốc, thay thế hoàn toàn cách xử lý phẳng, chuỗi thô của v1. Các điểm cập nhật cốt lõi bao gồm:

1. Mô hình dữ liệu có cấu trúc (`Reference.csl` thay vì chuỗi thô)

- **Bản cũ (v1):** Lưu `content: string` và danh mục tham khảo chỉ là các dòng text thô, trích dẫn trong bài là các ký tự gõ chết kiểu `[1, 2]`.
- **Bản mới (v2):**
  - Định nghĩa `interface Reference` có cấu trúc: `{ id: Id, raw: string, csl?: CslJson, doi?: string, url?: string, parse: 'ok' | 'partial' | 'failed' }`.
  - Dữ liệu chuẩn CSL-JSON (`Reference.csl`) trở thành dữ liệu chính, còn chuỗi `raw` chỉ dùng làm bản gốc để đối chiếu. Nhờ có dữ liệu cấu trúc này mới có thể tự động đổi kiểu trích dẫn (ví dụ từ Vancouver sang APA).

2. Trích dẫn nội dòng thành đối tượng (`Inline Node`), trỏ bằng ID ổn định

- **Không còn chuỗi `[1]` cứng:** Trong văn bản, trích dẫn là một node riêng: `{ t: 'cite', refIds: Id[] }`.
- Node này có thể mở rộng các thuộc tính cho phong cách tác giả-năm như: `locator` (trang trích dẫn), `prefix`, `suffix`, hoặc `suppressAuthor` (dành cho câu kiểu: *"Nguyễn (2020) cho thấy..."*).

3. Số thứ tự và nhãn trích dẫn là dữ liệu dẫn xuất (`Derived Data`)

- Không còn việc tự sửa tay số `[1]`, `[2]` hay dùng regex để renumber:
  - Hàm thuần túy `computeNumbering(doc, citationStyle)` sẽ tính toán toàn bộ thứ tự xuất hiện lần đầu (`citationOrder`), nhãn hiển thị trong bài (`citationLabels`), và thứ tự danh mục tài liệu tham khảo cuối bài (`bibliographyOrder`).
  - Khi xóa hoặc chèn thêm tài liệu tham khảo ở giữa bài, số thứ tự và nhãn tự động nhảy lại tức thì.
  - Tự động phát hiện các lỗi logic: tài liệu tham khảo không được nhắc tới (`uncitedRefs`), hoặc trích dẫn trỏ vào ID không tồn tại (`danglingCites`).

4. Hệ thống trích dẫn mở rộng được qua CSL (Citation Style Language)

- Kiểu trích dẫn được đưa vào **cấu hình (configuration)** của tạp chí, không phải code cứng trong phần mềm:
  - Hỗ trợ cả 2 nhóm phong cách lớn: **Numeric** (đánh số theo thứ tự xuất hiện như Vancouver, IEEE) và **Author-Year** (tác giả-năm, danh mục xếp theo alphabet như APA7).
  - Triển khai qua `CslCitationStyle` sử dụng `citeproc-js` cùng các file định dạng `.csl` có sẵn. Thêm kiểu trích dẫn mới chỉ cần thêm file `.csl` và khai báo trong `JournalProfile`, không phải sửa `Manuscript`, `Editor` hay `Exporters`.

- Khai báo hiển thị trong Profile: `citation: { style: 'apa7', locale: 'vi-VN', inText: 'bracket' | 'superscript' | 'parenthetical' }`.
- Hỗ trợ **Locale tiếng Việt** cho trích dẫn: tự động chuyển *"et al."* thành *"và cs."*, *"pp."* thành *"tr."*, và đảo họ tên tác giả tiếng Việt đúng chuẩn.

5. Cải tiến bóc tách Word (`ingest-docx`) cho Reference

- Đọc trực tiếp các trường trích dẫn tự động từ các phần mềm quản lý trích dẫn: xử lý thẻ `w:fldChar` / `w:instrText` của **Zotero, Mendeley** (`ADDIN CSL_CITATION`) và **EndNote** (`ADDIN EN.CITE`) để lấy thẳng dữ liệu có cấu trúc đưa vào `Reference.csl`.
- Nếu là file Word gõ tay: `AuthorYearCitationRecognizer` nhận diện các dạng `(Nguyễn, 2020)` để ghép với danh mục, và `ReferenceParser` phân tích chuỗi thô thành CSL.

6. Luật kiểm tra chất lượng riêng (QC Engine) cho Reference

Bộ luật QC tự động kiểm tra:

- Với kiểu đánh số (Vancouver): kiểm tra số thứ tự trích dẫn khớp đúng với thứ tự xuất hiện.
- Với kiểu tác giả-năm (APA): kiểm tra mỗi cite khớp duy nhất một reference; nếu trùng tác giả và năm thì cảnh báo cần hậu tố `a`, `b` (ví dụ `2020a`, `2020b`).
- Kiểm tra thiếu các trường bắt buộc mà style đòi hỏi (như APA cần đủ tác giả, năm, tựa đề).
- Cảnh báo các tài liệu có trạng thái bóc tách lỗi (`parse: 'partial'` hoặc `'failed'`) trước khi cho phép người dùng đổi style trích dẫn.

7. Xuất bản đồng nhất (Exporters)

- Chuỗi trích dẫn và danh mục được bộ thư viện CSL (`citeproc`) format sẵn từ trước, **không để BibTeX/biblatex tự ý định dạng**.
- Nhờ vậy, định dạng tài liệu tham khảo khi xuất ra **PDF, preview HTML, DOCX và JATS XML** (`<xref ref-type="bibr">`) luôn cho ra kết quả đồng nhất 100%.

User prompt: cái đuôi .csl là sao vậy

Response: Đuôi file `.csl` là viết tắt của **Citation Style Language** (Ngôn ngữ Định dạng Trích dẫn).

Đây là một định dạng file dạng XML được chuẩn hóa quốc tế, chuyên dùng để **mô tả các quy tắc trình bày trích dẫn và danh mục tài liệu tham khảo** trong giới học thuật.

1. File `.csl` giải quyết vấn đề gì?

Trước đây, mỗi tạp chí có một quy định trích dẫn khác nhau:

- **APA:** `(Nguyễn, 2020)` — danh mục xếp theo thứ tự bảng chữ cái A-Z, in nghiêng tên sách/tập san.
- **Vancouver / IEEE:** `[1]` hoặc dạng chỉ số trên — danh mục xếp theo thứ tự xuất hiện lần đầu trong bài.
- **Harvard, Chicago, Nature...** mỗi nơi một kiểu chấm, phẩy, ngoặc đơn, viết tắt *"et al."* khác nhau.

Nếu không có CSL, lập trình viên sẽ phải viết code logic rất phức tạp bằng tay cho từng tạp chí.

Với `.csl`, toàn bộ quy tắc hiển thị (dấu ngoặc tròn hay vuông, in đậm hay in nghiêng, tên tác giả đảo trước hay sau) được viết sẵn thành một file cấu hình chuẩn. Hiện tại cộng đồng học thuật toàn cầu đã xây dựng sẵn kho dữ liệu mở chứa hàng ngàn file `.csl` cho hầu hết các tạp chí khoa học trên thế giới (Zotero, Mendeley, Papers... đều dùng kho này).

2. File `.csl` đóng vai trò gì trong StructPaper v2?

Trong tài liệu đề xuất (đặc biệt là **Mục 3.5**), hệ thống áp dụng cơ chế: **Kiểu trích dẫn là cấu hình, không phải code**.

- **Dùng chung engine `citeproc-js`:** Thay vì viết code xử lý cho từng style, hệ thống nhúng bộ máy đọc CSL (`citeproc-js`).
- **Thêm hoặc đổi kiểu trích dẫn chỉ mất vài giây:**

    - Muốn đổi từ **Vancouver** sang **APA 7th** hay **IEEE**, nhóm không cần phải sửa code TypeScript, không đụng vào giao diện hay trình biên dịch LaTeX.
    - Bạn chỉ cần tải file `vancouver.csl` hoặc `apa-7th-edition.csl` thả vào hệ thống và khai báo tên file trong `JournalProfile`.
- **Tự động sinh kết quả đồng nhất:** Bộ máy đọc file `.csl` sẽ tự động định dạng danh mục tham khảo và nhãn trích dẫn trong bài, đảm bảo xuất ra màn hình soạn thảo, file PDF, file Word (`.docx`) hay JATS XML đều khớp nhau 100%.
